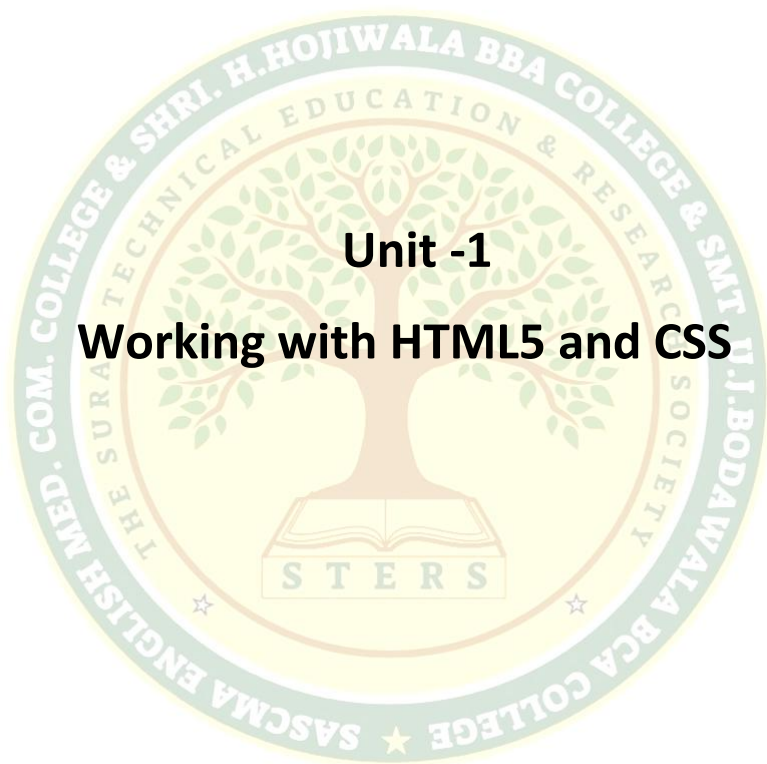


BACHELOR OF COMPUTER APPLICATIONS



Unit -1

Working with HTML5 and CSS

Index

1.1	Concepts of CSS:	1
	1.1.1 Adding CSS (Inline ,Internal, External)	2
	1.1.2 HTML Links and attribute (_self, _blank, _parent, _top)	4
	1.1.3 Absolute URL and Relative URL	6
	1.1.4 tag and its attributes (src, alt, style ,width ,height)	6
1.2	HTML forms:	7
	1.2.1 Form Elements and their attributes	7
	1.2.1.1 form (action, method, novalidate, autocomplete, target)	7
	1.2.1.2 label, input (text, radio button, Checkboxes, submit/reset button)	12
	1.2.1.3 select(id, name,value)	14
	1.2.1.4 textarea (name, rows, cols)	16
	1.2.1.5 button(type, onclick)	17
	1.2.1.6 datalist ☆	18
	1.2.2 Media : Video, Audio	19

1.1 Concepts of CSS:

What is CSS?

- CSS stands for Cascading Style Sheets,
- CSS describes how HTML elements are to be displayed on screen, paper, or in other media.
- CSS saves a lot of work. It can control the layout of multiple web pages all at once.
- External style sheets are stored in CSS files.

Why Use CSS?

CSS is used to define styles for your web pages, including the design, layout and variations in display for different devices and screen sizes.

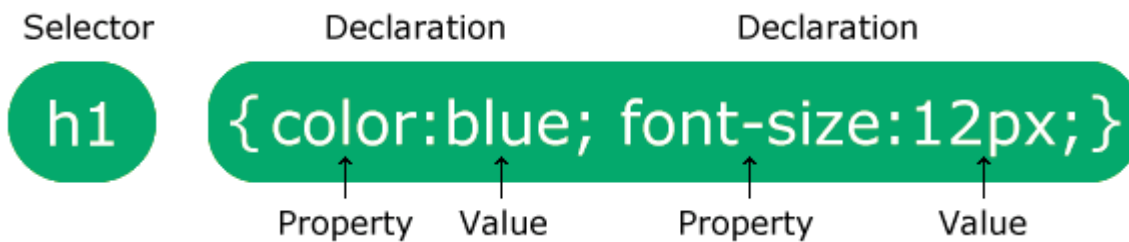
CSS Solved a Big Problem

- HTML was NEVER intended to contain tags for formatting a web page!
- HTML was created to describe the content of a web page, like:
- `<h1>This is a heading</h1>`
- `<p>This is a paragraph.</p>`
- When tags like `<font>`, and color attributes were added to the HTML 3.2 specification, it started a nightmare for web developers. Development of large websites, where fonts and color information were added to every single page, became a long and expensive process.
- To solve this problem, the World Wide Web Consortium (W3C) created CSS.
- CSS removed the style formatting from the HTML page!

CSS Saves a Lot of Work!

- The style definitions are normally saved in external .css files.
- With an external stylesheet file, you can change the look of an entire website by changing just one file!

CSS Syntax



- The selector points to the HTML element you want to style.
- The declaration block contains one or more declarations separated by semicolons.
- Each declaration includes a CSS property name and a value, separated by a colon.
- Multiple CSS declarations are separated with semicolons, and declaration blocks are surrounded by curly braces.

Example

In this example all `<p>` elements will be center-aligned, with a red text color:

```
<style>
```

```
p {
```

```
  color: red;
```

```
  text-align: center;
```

```
}
```

```
</style>
```

Example Explained

- `p` is a selector in CSS (it points to the HTML element you want to style: `<p>`).
- `color` is a property, and `red` is the property value
- `text-align` is a property, and `center` is the property value

1.1.1 Adding CSS (Inline, Internal, External)

1. Inline CSS

- An inline CSS is used to apply a unique style to a single HTML element.
- An inline CSS uses the `style` attribute of an HTML element.

- The following example sets the text color of the <h1> element to blue, and the text color of the <p> element to red:

Example:

```
<html>
<body>
<h1 style="color:blue;">A Blue Heading</h1>
<p style="color:red;">A red paragraph.</p>
</body>
</html>
```

2. Internal CSS

- An internal CSS is used to define a style for a single HTML page.
- An internal CSS is defined in the <head> section of an HTML page, within a <style> element.
- The following example sets the text color of ALL the <h1> elements (on that page) to blue, and the text color of ALL the <p> elements to red. In addition, the page will be displayed with a "powderblue" background color:

Example:

```
<html>
<head>
<style>
body {background-color: powderblue;}
h1 {color: blue;}
p {color: red;}
</style>
</head>
<body>
<h1>This is a heading</h1>
<p>This is a paragraph.</p>
</body>
</html>
```

3. External CSS

- An external style sheet is used to define the style for many HTML pages.
- To use an external style sheet, add a link to it in the <head> section of each HTML page:

Example

```
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="mystyles.css">
</head>
<body>
  <h1>This is a heading</h1>
  <p>This is a paragraph.</p>
</body>
</html>
```

1.1.2 HTML Links and attributes

HTML Links - Hyperlinks

- HTML links are hyperlinks.
- You can click on a link and jump to another document.
- When you move the mouse over a link, the mouse arrow will turn into a little hand.

Syntax

- The HTML <a> tag defines a hyperlink. It has the following syntax:

```
<a href="url">link text</a>
```

- The most important attribute of the <a> element is the href attribute, which indicates the link's destination.
- The link text is the part that will be visible to the reader.

- Clicking on the link text, will send the reader to the specified URL address.

Example

```
<html>

<body>

<h1>HTML Links</h1>

<p><a href="https://www.google.com/">Visit Google Website</a></p>

</body>

</html>
```

HTML Links - The target attribute

- By default, the linked page will be displayed in the current browser window. To change this, you must specify another target for the link.
 - The target attribute specifies where to open the linked document.
 - The target attribute can have one of the following values:
1. `_self` - Default. Opens the document in the same window/tab as it was clicked
 2. `_blank` - Opens the document in a new window or tab
 3. `_parent` - Opens the document in the parent frame
 4. `_top` - Opens the document in the full body of the window

Example

```
<html>

<body>

<h2>The target Attribute</h2>

<a href="https://www.google.com/" target="_blank">Visit Google!</a>

</body>

</html>
```

1.1.3 Absolute URL and Relative URL in <href>

Absolute URL

An absolute link is a hyperlink containing a full URL, which includes all the information needed to find a particular site, page or document or other addressable item on the Internet. This information includes: The protocol to use, such as HTTP (Hypertext Transfer Protocol) or FTP (File Transfer Protocol).

Relative URL

Relative hyperlinks are addresses that are relative to the current domain or location. They only contain the name of the target page prefixed with any necessary folder moves (for example, default. html)

Example

```
<html>
<body>
<h2>Absolute URLs</h2>
<p><a href="https://www.w3.org/">W3C</a></p>
<p><a href="https://www.google.com/">Google</a></p>
<h2>Relative URLs</h2>
<p><a href="images_bs.html">HTML Images</a></p>
<p><a href="button.html">CSS Tutorial</a></p>
</body>
</html>
```

1.1.4 tag and its attributes

- The tag is used to embed an image in an HTML page.
- Images are not technically inserted into a web page; images are linked to web pages. The tag creates a holding space for the referenced image.
- The tag has two required attributes:
 1. src - Specifies the path to the image

2. alt - Specifies an alternate text for the image, if the image for some reason cannot be displayed

Note: Also, always specify the width and height of an image. If width and height are not specified, the page might flicker while the image loads.

Tip: To link an image to another document, simply nest the tag inside an [<a>](#) tag (see example below).

Example

```
<html>

<body>



</body>

</html>
```

1.2 HTML Forms:

An HTML form is used to collect user input. The user input is most often sent to a server for processing.

<form>

.

form elements

.

</form>

The <form> element is a container for different types of input elements, such as: text fields, checkboxes, radio buttons, submit buttons, etc.

1.2.1 Form Elements and their attributes:

1.2.1.1 Form (action, Method, novalidate, autocomplete, target)

❖ The Action Attribute

- The action attribute defines the action to be performed when the form is submitted.

- Usually, the form data is sent to a file on the server when the user clicks on the submit button.
- In the example below, the form data is sent to a file called "home.html". This file contains a server-side script that handles the form data:

```
<html>

<body>

<h2>HTML Forms</h2>

<form action="home.html">

  <label for="fname">First name:</label><br>

  <input type="text" id="fname" name="fname" value="John"><br>

  <label for="lname">Last name:</label><br>

  <input type="text" id="lname" name="lname" value="Doe"><br><br>

  <input type="submit" value="Submit">

</form>

<p>If you click the "Submit" button, the form-data will be sent to a page called
"home.html".</p>

</body>

</html>
```

❖ The Method Attribute

- The method attribute specifies the HTTP method to be used when submitting the form data.
- The form-data can be sent as URL variables (with method="get") or as HTTP post transaction (with method="post").
- The default HTTP method when submitting form data is GET.

Example

```
<html>

<body>

<h2>The method Attribute</h2>
```

```
<p>This form will be submitted using the GET method:</p>
<form action="home.html" target="_blank" method="get">
  <label for="fname">First name:</label><br>
  <input type="text" id="fname" name="fname" value="John"><br>
  <label for="lname">Last name:</label><br>
  <input type="text" id="lname" name="lname" value="Doe"><br><br>
  <input type="submit" value="Submit">
</form>

<p>After you submit, notice that the form values is visible in the address bar of
the new browser tab.</p>
</body>
</html>
```

❖ novalidate attribute:

The novalidate attribute is a Boolean attribute. When present, it specifies that the form-data (input) should not be validated when submitted.

Example:

```
<html>
<body>
<h1>The form novalidate attribute</h1>
<form action="home.html" novalidate>
  <label for="email">Enter your email:</label>
  <input type="email" id="email" name="email"><br><br>
  <input type="submit">
</form>
```

<p>Note: The novalidate attribute of the form tag is not supported in Safari 10 (or earlier).</p>

</body>

</html>

❖ autocomplete attribute

The autocomplete attribute specifies whether or not an input field should have autocomplete enabled. Autocomplete allows the browser to predict the value. When a user starts to type in a field, the browser should display options to fill in the field, based on earlier typed values.

Note: The autocomplete attribute works with the following input types: text, search, url, tel, email, password, datepickers, range, and color.

Example

<html>

<body>

<h1>The autocomplete attribute</h1>

<p>Fill in and submit the form, then reload the page to see how autocomplete works.</p>

<p>Notice that autocomplete is "on" for the form, but "off" for the e-mail field!</p>

<form action="home.html" autocomplete="on">

<label for="fname">First name:</label>

<input type="text" id="fname" name="fname">

<label for="lname">Last name:</label>

<input type="text" id="lname" name="lname">

<label for="email">Email:</label>

<input type="email" id="email" name="email" autocomplete="off">

<input type="submit">

</form>

```
</body>
```

```
</html>
```

❖ target attribute

The target attribute specifies a name or a keyword that indicates where to display the response that is received after submitting the form.

The target attribute defines a name of, or keyword for, a browsing context (e.g. tab, window, or inline frame).

Example

```
<html>
```

```
<body>
```

```
<h1>The form target attribute</h1>
```

```
<form action="home.html" method="get" target="_blank">
```

```
<label for="fname">First name:</label>
```

```
<input type="text" id="fname" name="fname"><br><br>
```

```
<label for="lname">Last name:</label>
```

```
<input type="text" id="lname" name="lname"><br><br>
```

```
<input type="submit" value="Submit">
```

```
</form>
```

```
</body>
```

```
</html>
```

➤ Attribute Values

Value	Description
<code>_blank</code>	The response is displayed in a new window or tab
<code>_self</code>	The response is displayed in the same frame (this is default)

_parent	The response is displayed in the parent frame
_top	The response is displayed in the full body of the window

1.2.1.2 label, input (text, radio, button, Checkboxes, submit/reset)

❖ label

The <label> tag defines a label for several elements: <input>, <meter>, <progress>, <select>, <textarea>, etc.

Example

```
<html>
<body>
<h1>The label element</h1>
<p>Click on one of the text labels to toggle the related radio button:</p>
<form action="home.html">
  <input type="radio" id="html" name="fav_language" value="HTML">
  <label for="html">HTML</label><br>
  <input type="radio" id="css" name="fav_language" value="CSS">
  <label for="css">CSS</label><br>
  <input type="radio" id="javascript" name="fav_language" value="JavaScript">
  <label for="javascript">JavaScript</label><br><br>
  <input type="submit" value="Submit">
</form>
</body>
</html>
```

❖ input

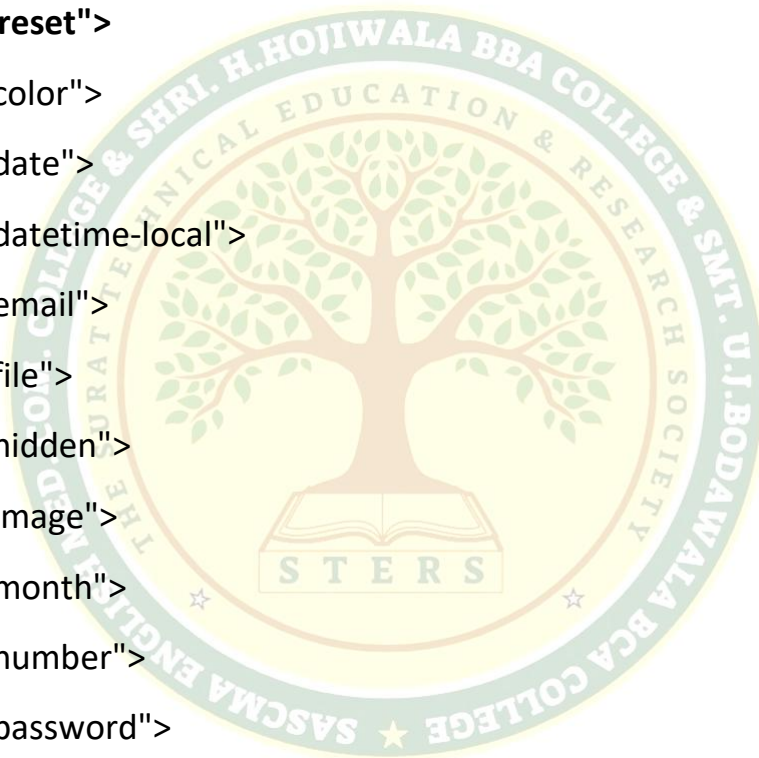
The <input> tag specifies an input field where the user can enter data.

The <input> element is the most important form element.

The `<input>` element can be displayed in several ways, depending on the type attribute.

The different input types are as follows:

- `<input type="text">` (default value)
- `<input type="radio">`
- `<input type="button">`
- `<input type="checkbox">`
- `<input type="submit">`
- `<input type="reset">`
- `<input type="color">`
- `<input type="date">`
- `<input type="datetime-local">`
- `<input type="email">`
- `<input type="file">`
- `<input type="hidden">`
- `<input type="image">`
- `<input type="month">`
- `<input type="number">`
- `<input type="password">`
- `<input type="range">`
- `<input type="search">`
- `<input type="tel">`
- `<input type="time">`
- `<input type="url">`
- `<input type="week">`



Note: Always use the [<label>](#) tag to define labels for <input type="text">, <input type="checkbox">, <input type="radio">, <input type="file">, and <input type="password">.

Example

```
<html>

<body>

<h1>The input element</h1>

<form action="home.html">

  <label for="fname">First name:</label>

  <input type="text" id="fname" name="fname"><br><br>

  <label for="lname">Last name:</label>

  <input type="text" id="lname" name="lname"><br><br>

  <input type="submit" value="Submit">

</form>

<p>Click the "Submit" button and the form-data will be sent to a page on the
server called "home.html".</p>

</body>

</html>
```

1.2.1.3 select (id, name, <option>)

The <select> element is used to create a drop-down list.

The <select> element is most often used in a form, to collect user input.

The name attribute is needed to reference the form data after the form is submitted (if you omit the name attribute, no data from the drop-down list will be submitted).

The id attribute is needed to associate the drop-down list with a label.

The [<option>](#) tags inside the <select> element define the available options in the drop-down list.

Example

```
<html>

<body>

<h1>The select element</h1>

<p>The select element is used to create a drop-down list.</p>

<form action="home.html">

  <label for="cars">Choose a car:</label>

  <select name="cars" id="cars">

    <option value="volvo">Volvo</option>

    <option value="bmw">Saab</option>

    <option value="mercedes">Opel</option>

    <option value="audi">Audi</option>

  </select>

  <br><br>

  <input type="submit" value="Submit">

</form>

<p>Click the "Submit" button and the form-data will be sent to a page on the
server called "home.html".</p>

</body>

</html>
```

Example (group option)

```
<html>

<body>

<h1>The optgroup element</h1>
```

<p>The optgroup tag is used to group related options in a drop-down list:</p>

```
<form action="home.html">
```

```
<label for="cars">Choose a car:</label>
```

```
<select name="cars" id="cars">
```

```
<optgroup label="Swedish Cars">
```

```
<option value="volvo">Volvo</option>
```

```
<option value="bmw">Saab</option>
```

```
</optgroup>
```

```
<optgroup label="German Cars">
```

```
<option value="mercedes">Mercedes</option>
```

```
<option value="audi">Audi</option>
```

```
</optgroup>
```

```
</select>
```

```
<br><br>
```

```
<input type="submit" value="Submit">
```

```
</form>
```

```
</body>
```

```
</html>
```

1.2.1.4 textarea

The <textarea> tag defines a multi-line text input control.

The <textarea> element is often used in a form, to collect user inputs like comments or reviews.

A text area can hold an unlimited number of characters, and the text renders in a fixed-width font (usually Courier).

The size of a text area is specified by the <cols> and <rows> attributes (or with CSS).

The **name** attribute is needed to reference the form data after the form is submitted (if you omit the name attribute, no data from the text area will be submitted).

The id attribute is needed to associate the text area with a label.

Example

```
<html>
<body>
<h1>The textarea element</h1>
<form action="home.html">
  <p><label for="review">Review of Google:</label></p>
  <textarea id="review" name="review" rows="4" cols="50">At google.com you will
  learn how to make a website. They offer free tutorials in all web development
  technologies.</textarea>
  <br>
  <input type="submit" value="Submit">
</form>
<p>Click the "Submit" button and the form-data will be sent to a page on the
server called "home.html".</p>
</body>
</html>
```

1.2.1.5 button

The <button> tag defines a clickable button.

Inside a <button> element you can put text (and tags like <i>, , ,
, , etc.). That is not possible with a button created with the [<input>](#) element!

Note: Always specify the type attribute for a <button> element, to tell browsers what type of button it is.

Example

```
<html>
<body>
<h1>The button Element</h1>
<button type="button" onclick="alert('Hello world!')">Click Me!</button>
</body>
</html>
```

1.2.1.6 datalist

The <datalist> tag specifies a list of pre-defined options for an <input> element.

The <datalist> tag is used to provide an "autocomplete" feature for <input> elements. Users will see a drop-down list of pre-defined options as they input data.

The <datalist> element's id attribute must be equal to the <input> element's list attribute (this binds them together).

Example

```
<html>
<body>
<h1>The datalist element</h1>
<form action="home.html" method="get">
  <label for="browser">Choose your browser from the list:</label>
  <input list="browsers" name="browser" id="browser">
  <datalist id="browsers">
    <option value="Edge">
    <option value="Firefox">
    <option value="Chrome">
    <option value="Opera">
    <option value="Safari">
  </datalist>
```

```
<input type="submit">
</form>
<p><strong>Note:</strong> The datalist tag is not supported in Safari 12.0 (or
earlier).</p>
</body>
</html>
```

1.2.2 Media: Video, Audio

What is Multimedia?

Multimedia comes in many different formats. It can be almost anything you can hear or see, like images, music, sound, videos, records, films, animations, and more.

Web pages often contain multimedia elements of different types and formats.

Multimedia elements (like audio or video) are stored in media files.

The most common way to discover the type of a file, is to look at the file extension.

Multimedia files have formats and different extensions like: .wav, .mp3, .mp4, .mpg, .wmv, and .avi.

HTML Video

The HTML <video> element is used to show a video on a web page.

The controls attribute adds video controls, like play, pause, and volume.

It is a good idea to always include width and height attributes. If height and width are not set, the page might flicker while the video loads.

The <source> element allows you to specify alternative video files which the browser may choose from. The browser will use the first recognized format.

<source> Defines multiple media resources for media elements, such as <video> and <audio>

The text between the <video> and </video> tags will only be displayed in browsers that do not support the <video> element.

Add muted after autoplay to let your video start playing automatically (but muted):

Example

```
<html>
<body>
<video width="320" height="240" autoplay>
  <source src="movie.mp4" type="video/mp4">
  Your browser does not support the video tag.
</video>
</body>
</html>
```

HTML Audio

The HTML <audio> element is used to play an audio file on a web page.

How It Works

The controls attribute adds audio controls, like play, pause, and volume.

The <source> element allows you to specify alternative audio files which the browser may choose from. The browser will use the first recognized format.

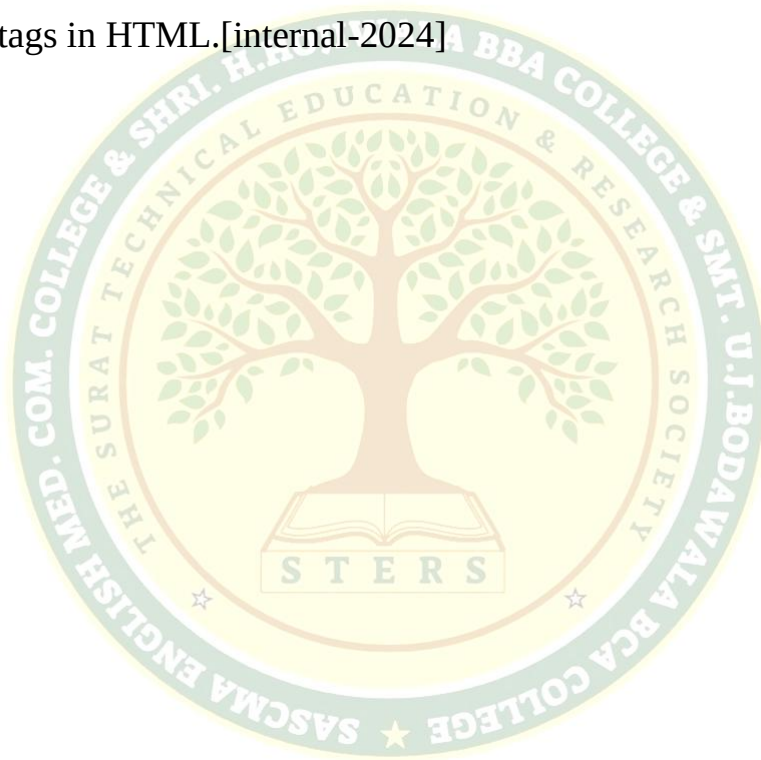
The text between the <audio> and </audio> tags will only be displayed in browsers that do not support the <audio> element.

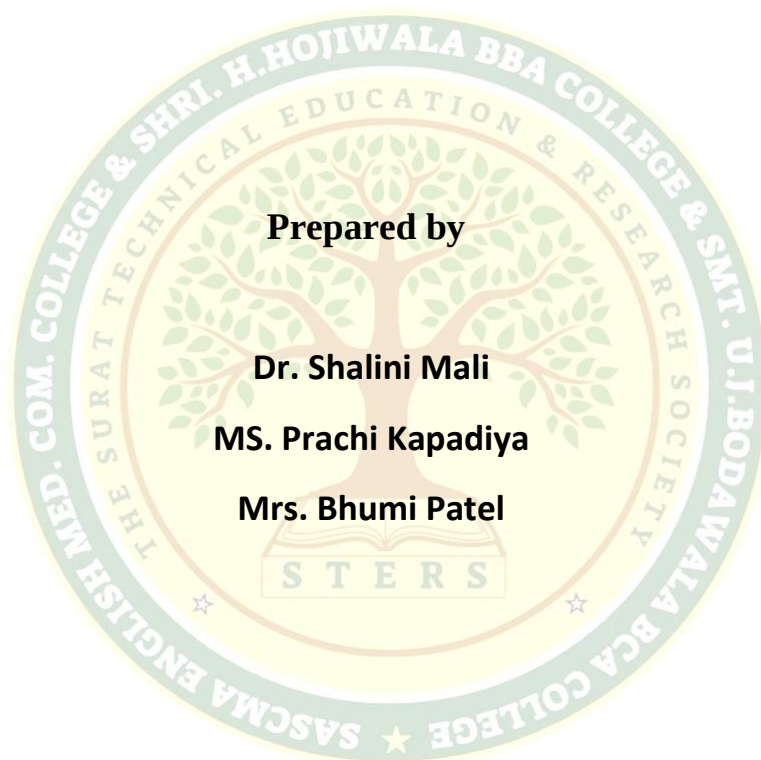
Example

```
<html>
<body>
<audio controls autoplay>
  <source src="horse.mp3" type="audio/mpeg">
  Your browser does not support the audio element.
</audio>
</body>
</html>
```

Previous Years' Veer Narmad South Gujarat University Examination Questions

- 1.Explain different From Elements along with their attributes.[nov-2024]
- 2.Explain different ways to adding CSS. Give their advantages and disadvantages.
[nov-2024]
- 3.what is CSS? Explain it's types with examples.[march-2025]
- 4.Expalin audio and video tag with examples.[march-2025]
5. Explain media tags in HTML.[internal-2024]





Unit-2: Introduction of Bootstrap

Unit-2: Design website using Bootstrap 4
[As per VNSGU Syllabus] – Year 2025-2026



Index

	Content	Page No
2.1	Bootstrap Introduction	
2.2	Grid Structure	
2.3	Table, colour, Alert, Form controls	
2.4	Button and Button group	
2.5	Images, Media Object	
2.6	Pagination	
2.7	Bootstrap Grids	
2.8	Bootstrap theme	



2.1 Bootstrap Introduction

- Bootstrap is a free front-end, open-source CSS framework developed by Twitter for creating responsive and mobile-first websites.
- It provides ready-made CSS classes and JavaScript components that help developers build websites quickly.
- Bootstrap includes HTML and CSS based design templates for typography, forms, buttons, tables, navigation, modals, image carousels and many other, as well as optional JavaScript plugins.

What is Responsive Web Design?

- Responsive web design is about creating web sites which automatically adjust themselves to look good on all devices, from small phones to large desktop.

HTML vs Bootstrap4

HTML	Bootstrap4
Basic Webpage Structure	Ready-made styling framework
Manual CSS required	Predefined CSS Classes
Not responsive by default	Responsive by default
Takes more coding	Less coding

Advantages of Bootstrap:

- **Easy to use:** Anybody with just basic knowledge of HTML and CSS can start using Bootstrap
- **Responsive features:** Bootstrap's responsive CSS adjusts to phones, tablets, and desktops
- **Mobile-first approach:** In Bootstrap, mobile-first styles are part of the core framework
- **Browser compatibility:** Bootstrap 4 is compatible with all modern browsers (Chrome, Firefox, Internet Explorer 10+, Edge, Safari, and Opera)

Where to Get Bootstrap 4?

There are two ways to start using Bootstrap 4 on your own web site.

1. Include Bootstrap 4 from a CDN

```
<link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@4.6.2/dist/css/bootstrap.min.css">
```

One advantage of using the Bootstrap 4 CDN:

- Many users already have downloaded Bootstrap 4 from jsdelivr when visiting another site. As a result, it will be loaded from cache when they visit your site, which leads to faster loading time. Also, most CDN's will make sure that once a user requests a file from it, it will be served from the server closest to them, which also leads to faster loading time.

2. Download Bootstrap 4 from getbootstrap.com

- If you want to download and host Bootstrap 4 yourself, go to <https://getbootstrap.com/>, and follow the instructions there.
- Add the HTML5 doctype
 - Bootstrap 4 uses HTML elements and CSS properties that require the HTML5 doctype.
 - Always include the HTML5 doctype at the beginning of the page, along with the lang attribute and the correct character set:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
  </head>
</html>
```
- Bootstrap 4 is mobile-first
- Bootstrap 4 is designed to be responsive to mobile devices. Mobile-first styles are part of the core framework.
- To ensure proper rendering and touch zooming, add the following <meta> tag inside the <head> element:

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

width=device-width	Sets the width of the page to follow the screen-width of the device (which will vary depending on the device).
initial-scale=1	Sets the initial zoom level when the page is first loaded by the browser

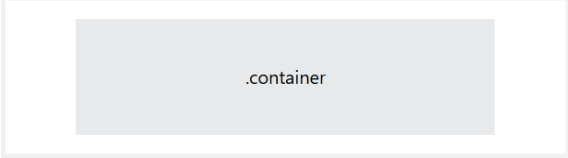
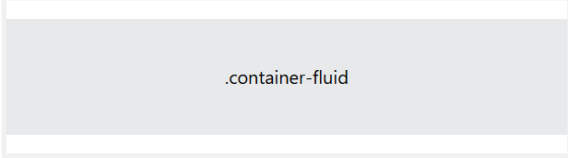
3. Add Container, Row and Column

Bootstrap4 Container

Bootstrap 4 also requires a containing element to wrap site contents.

There are **two container** classes to choose from:

1. The .container class provides a responsive **fixed width container**
2. The .container-fluid class provides a **full width container**, spanning the entire width of the viewport

Container	Container-fluid
Fixed width	Full screen width
Leaves Margin	Covers complete screen
Best for website	Best for dashboards
	

2.2 Grid Structure

- Bootstrap's grid system is built with flexbox and allows developers to create responsive layouts that adapt to various devices, from mobile phones to large desktops.
- It uses a **12-column structure**, where columns can be combined or sized individually to create flexible layouts.
- Core Components
 - Containers: Wrap the content and provide horizontal padding. Use `.container` for a responsive fixed-width layout or `.container-fluid` for full-width layouts Bootstrap+1.
 - Rows: Serve as horizontal wrappers for columns. Rows use negative margins to offset column padding (gutters) and ensure proper alignment.
 - Columns: Define the width of content. Columns are created using classes like `col`, `.col-*`, or `.col-*-*`, where the first `*` indicates the breakpoint (e.g., `sm`, `md`, `lg`) and the second `*` specifies the number of columns (1–12),
- Responsive Breakpoints
Bootstrap 5 supports six responsive breakpoints:
 - xs (extra small, <576px)
 - sm (small, ≥576px)
 - md (medium, ≥768px)
 - lg (large, ≥992px)
 - xl (extra large, ≥1200px)
 - xxl (extra extra large, ≥1400px)

Columns can be customized for each breakpoint, allowing layouts to stack on smaller screens and display horizontally on larger screens

span 1	span 1	span 1	span 1	span 1	span 1	span 1	span 1	span 1	span 1	span 1	span 1
span 4				span 4				span 4			
span 4				span 8							
span 6						span 6					
span 12											

Column Behaviour

- Columns in a row should add up to 12; if they exceed 12, they will wrap to the next line.
- Use `col` for equal-width columns or `col-*-*` to specify exact widths.
- Columns automatically create gutters (spacing between columns) via padding, which is counteracted by row margins.

- You can mix different column classes for different breakpoints to create dynamic layouts, e.g., .col-12 .col-md-8 for full-width on mobile and 8/12 width on medium screens

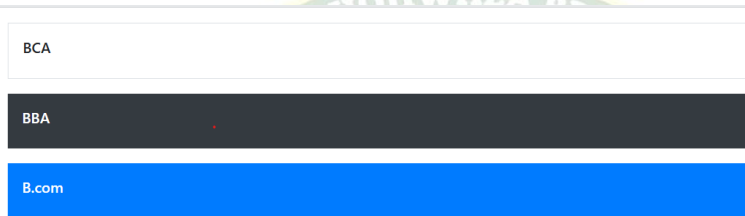
<pre><div class="container"> <div class="row"> <div class="col -6 bg-danger"> Left </div> <div class="col -6 bg-danger"> Right </div> </div> </div></pre>	<pre><div class="container"> <div class="row"> <div class="col -4 bg-danger"> Left </div> <div class="col -4 bg-info"> Right </div> <div class="col -4 bg-success"> Right </div> </div> </div></pre>
---	--

Container Padding	By default, containers have 15px left and right padding, with no top or bottom padding. Therefore, we often use spacing utilities, such as extra padding and margins to make them look even better. For example, .pt-3 means "add a top padding of 16px": <div class="container pt-3"></div>	BCA Bachelor of Computer Application SASCMA,VNSGU,SURAT
	Code: <pre><!DOCTYPE html> <html lang="en"> <head> <title>Bootstrap Example</title> <meta charset="utf-8"> <meta name="viewport" content="width=device-width, initial-scale=1"> <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@4.6.2/dist/css/bootstrap.min.css"> <body> <div class="container pt-3"> <h1>BCA</h1> <p>Bachelor of Computer Application</p> <p>SASCMA,VNSGU,SURAT</p> </div> </body> </html></pre>	
	<pre><!DOCTYPE html> <html lang="en"> <head> <meta charset="utf-8"></pre>	

```

<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@4.6.2/dist/css/bootstrap.min.css">
<body>
<div class="container p-3 my-3 border">
  <h6>BCA</h6>
</div>
<div class="container p-3 my-3 bg-dark text-white">
  <h6>BBA</h6>
</div>
<div class="container p-3 my-3 bg-primary text-white">
  <h6>B.com</h6>
</div>
</body>
</html>

```



Class	Full Form	One-Line Description
m	Margin	Space outside all sides
mt	Margin Top	Space outside top
mb	Margin Bottom	Space outside bottom
ml	Margin Left	Space outside left
mr	Margin Right	Space outside right
mx	Margin X-axis	Space outside left & right
my	Margin Y-axis	Space outside top & bottom
p	Padding	Space inside all sides
pt	Padding Top	Space inside top
pb	Padding Bottom	Space inside bottom
pl	Padding Left	Space inside left
pr	Padding Right	Space inside right
px	Padding X-axis	Space inside left & right
py	Padding Y-axis	Space inside top & bottom

2.3 Table , Colour , Alert , Form Controls

Bootstrap 4 table has a light padding and horizontal dividers. The .table class adds basic styling to a table:

```

<div class="container">
  <table class="table">
    <thead>
      <tr>
        <th>Firstname</th>
        <th>Lastname</th>
      </tr>
    </thead>
  </table>
</div>

```

```

</thead>
<tbody>
  <tr>
    <td>XYZ</td>
    <td>ROCKS</td>
  </tr>
  <tr>
    <td>ABC</td>
    <td>Champ</td>
  </tr>
</tbody>
</table>
</div>

```

Firstname	Lastname
XYZ	ROCKS
ABC	Champ

Class	Purpose
.table	Basic bootstrap table
.table-bordered	Add border to table
.table-hover	Highlight row on hover - adds a hover effect (grey background colour) on table rows
.table-striped	adds zebra-stripes to a table
.table-dark	adds a black background to the table
.table table-dark table-striped	create a dark, striped table
.table-borderless	removes borders from the table
.table-primary	Blue: Indicates an important action
.table-success	Green: Indicates a successful or positive action
.table-danger	Red: Indicates a dangerous or potentially negative action
.table-info	Light blue: Indicates a neutral informative change or action
.table-warning	Orange: Indicates a warning that might need attention
.table-active	Grey: Applies the hover color to the table row or table cell
.table-secondary	Grey: Indicates a slightly less important action
.table-light	Light grey table or table row background
.table-dark	Dark grey table or table row background
<thead class="thead-dark">	adds a black background to table headers
<thead class="thead-light">	class adds a grey background to table headers
.table-sm	class makes the table smaller by cutting cell padding in half

.table-responsive	class adds a scrollbar to the table when needed (when it is too big horizontally)
-------------------	---

Text Colors

The classes for text colors are: .text-muted, .text-primary, .text-success, .text-info, .text-warning, .text-danger, .text-secondary, .text-white, .text-dark, .text-body (default body color/often black) and .text-light.

```
<div class="container">
  <p class="text-muted">This text is muted.</p>
  <p class="text-primary">This text is important.</p>
  <p class="text-success">This text indicates success.</p>
  <p class="text-info">This text represents some information.</p>
  <p class="text-warning">This text represents a warning.</p>
  <p class="text-danger">This text represents danger.</p>
  <p class="text-secondary">Secondary text.</p>
  <p class="text-dark">This text is dark grey.</p>
  <p class="text-body">Default body color (often black).</p>
  <p class="text-light">This text is light grey (on white background).</p>
  <p class="text-white">This text is white (on white background).</p>
</div>
```

This text is muted.

This text is important.

This text indicates success.

This text represents some information.

This text represents a warning.

This text represents danger.

Secondary text.

Dark grey text.

Body text.

```
<p class="text-black-50">Black text with 50% opacity on white background</p>
```

```
<p class="text-white-50 bg-dark">White text with 50% opacity on black background</p>
```

```
<p class="bg-primary text-white">This text is important.</p>
```

```
<p class="bg-success text-white">This text indicates success.</p>
```

Black text with 50% opacity on white background

White text with 50% opacity on black background

This text is important.

This text indicates success.

Alert:

- Bootstrap 4 provides an easy way to create predefined alert messages.
- Alerts are created with the .alert class, followed by one of the contextual classes .alert-success, .alert-info, .alert-warning, .alert-danger, .alert-primary, .alert-secondary, .alert-light or .alert-dark.

```
<div class="alert alert-success">
```

```
  <strong>Success!</strong> This alert box could indicate a successful or positive action.
```

```
</div>
```

```
<div class="alert alert-success">
```

```
  <strong>Success!</strong> You should <a href="#" class="alert-link">read this message</a>.
```

```
</div>
```

```
<div class="alert alert-success alert-dismissible">
```

```
  <button type="button" class="close" data-dismiss="alert">&times;</button>
```

```
  <strong>Success!</strong> This alert box could indicate a successful or positive action.
```

```
</div>
```

```
<div class="alert alert-success alert-dismissible fade show">
```

```
  <button type="button" class="close" data-dismiss="alert">&times;</button>
```

```
  <strong>Success!</strong> This alert box could indicate a successful or positive action.
```

```
</div>
```

Success! This alert box could indicate a successful or positive action.

Success! You should [read this message](#).

Success! This alert box could indicate a successful or positive action.

×

Success! This alert box could indicate a successful or positive action.

×

Form-Control:

- Form controls automatically receive some global styling with Bootstrap: All textual `<input>`, `<textarea>`, and `<select>` elements with class .form-control have a width of 100%.
- Bootstrap provides two types of form layouts:
 - Stacked (full-width) form
 - Inline form

Stacked(full-width) form :

Example :

Stacked form

Email:

Password:

☐ Remember me

```

<!DOCTYPE html>
<html lang="en">
<head>
  <title>Bootstrap Example</title>
  <meta charset="utf-8">
  <meta name="viewport"
content="width=device-width, initial-scale=1">
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@4.6
.2/dist/css/bootstrap.min.css">
</head>
<body>
<div class="container">
  <h2>Stacked form</h2>
  <form action="/action_page.php">
    <div class="form-group">
      <label for="email">Email:</label>
      <input type="email" class="form-control"
id="email" placeholder="Enter email"
name="email">
    </div>

```

```

<div class="form-group">
  <label for="pwd">Password:</label>
  <input type="password" class="form-control"
id="pwd" placeholder="Enter password"
name="pswd">
</div>
<div class="form-group form-check">
  <label class="form-check-label">
    <input class="form-check-input"
type="checkbox" name="remember"> Remember
me
  </label>
</div>
<button type="submit" class="btn btn-
primary">Submit</button>
</form>
</div>
</body>
</html>

```

Inline Form

Example:

Inline form

Email: Password: ☐ Remember me

```

<!DOCTYPE html>
<html lang="en">
<head>
  <title>Bootstrap Example</title>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-
width, initial-scale=1">
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@4.6.2/
dist/css/bootstrap.min.css">
<body>
<div class="container">
  <h2>Inline form</h2>
  <form class="form-inline"
action="/action_page.php">
    <label for="email">Email:</label>
    <input type="email" class="form-control"
id="email" placeholder="Enter email" name="email">
    <label for="pwd">Password:</label>
    <input type="password" class="form-control"
id="pwd" placeholder="Enter password"
name="pswd">

```

```

<div class="form-check">
  <label class="form-check-label">
    <input class="form-check-input"
type="checkbox" name="remember">
    Remember me
  </label>
</div>
<button type="submit" class="btn btn-
primary">Submit</button>
</form>
</div>
</body>
</html>

```

- If you want less grid margins (override default column gutters), use .form-row instead of .row.

2.4 Button and ButtonGroups

Bootstrap 4 provides different styles of buttons:

Basic **Primary** Secondary Success Info Warning Danger Dark Light Link

- If you want to give size to button use small , medium and large class like:

```
<button type="button" class="btn btn-primary btn-lg">Large</button>
```

```
<button type="button" class="btn btn-primary btn-md">Default</button>
```

```
<button type="button" class="btn btn-primary btn-sm">Small</button>
```

- .btn-block to create a block level button that spans the entire width of the parent element.
<button type="button" class="btn btn-primary btn-block">Button 1</button>
- If you want to give outline to button – [Bootstrap4 provides 8 outline class]
<button type="button" class="btn btn-outline-primary">Primary</button>

Primary Secondary Success Info Warning Danger Dark Light

```
<a href="#" class="btn btn-info">Link Button </a>
```

- A button can be set to an active (appear pressed) or a disabled (unclickable) state:

```
<button type="button" class="btn btn-primary active">Active Primary</button>
<button type="button" class="btn btn-primary" disabled>Disabled Primary</button>
```

Active Primary

Disabled Primary

The class `.active` makes a button appear pressed, and the `disabled` attribute makes a button unclickable. Note that `<a>` elements do not support the `disabled` attribute and must therefore use the `.disabled` class to make it visually appear disabled.

Button Groups

Bootstrap 4 allows you to group a series of buttons together (on a single line) in a button group: Use a `<div>` element with class `.btn-group` to create a button group.

```
<div class="btn-group">
  <button type="button" class="btn btn-primary">Ok</button>
  <button type="button" class="btn btn-primary">Close</button>
  <button type="button" class="btn btn-primary">Cancel</button>
</div>
```

The `.btn-group` class creates a button group:

Ok Close Cancel

```
<div class="btn-group-vertical">
  <button type="button" class="btn btn-primary">Ok </button>
  <button type="button" class="btn btn-primary">Close</button>
  <button type="button" class="btn btn-primary">Cancel</button>
</div>
```

Ok
Close
Cancel

Nesting buttongroup and dropdown list

```
<div class="btn-group">
  <button type="button" class="btn btn-primary">Apple</button>
  <button type="button" class="btn btn-primary">Samsung</button>
  <div class="btn-group">
    <button type="button" class="btn btn-primary dropdown-toggle" data-
toggle="dropdown">
      Sony
    </button>
  </div>
</div>
```

Nesting Button Groups

Nest button groups to create dropdown menus:

Apple Samsung Sony ▼
Tablet
Smartphone


```

<a class="dropdown-
item" href="#">Tablet</a>
<a class="dropdown-
item" href="#">Smartphone</a>
</div>
</div>
</div>

```

2.5 Image,Media object

Bootstrap 4 provides several classes to style and control images without writing custom CSS.

Class	Description	Example
.img-fluid	Makes the image responsive. Width becomes 100% of parent while maintaining aspect ratio.	<code></code>
.img-thumbnail	Adds a border, padding, and rounded corners to create a thumbnail effect.	<code></code>
.rounded	Gives the image slightly rounded corners.	<code></code>
.rounded-circle	Displays the image as a circle (best with square images).	<code></code>
Rounded Corners:  Circle:  Thumbnail: 		
.float-left	Floats the image to the left.	<code></code>
.float-right	Floats the image to the right.	<code></code>
.mx-auto	Centres an element horizontally (requires d-block). Automatic left and right margins.	<code></code>
.d-block	Makes the image a block-level element. Useful for centering.	<code></code>

Media Object

Bootstrap provides an easy way to align media objects (like images or videos) together with content. Media objects are often used to display blog comments, tweets and so on:

To create a media object, add the .media class to a container element, and place media content inside a child container with the .media-body class.

2.6 Pagination

If you have a website with lots of pages, you may wish to add some sort of pagination to each page. To create a basic pagination, add the .pagination class to an element. Then add the .page-item to each element and a .page-link class to each link inside .



```
<ul class="pagination">
  <li class="page-item"><a class="page-link" href="#">Previous</a></li>
  <li class="page-item"><a class="page-link" href="#">1</a></li>
  <li class="page-item"><a class="page-link" href="#">2</a></li>
  <li class="page-item"><a class="page-link" href="#">3</a></li>
  <li class="page-item"><a class="page-link" href="#">Next</a></li>
</ul>
```

- If you want to highlight current page use. active class

```
<li class="page-item active"><a class="page-link" href="#">2</a></li>
```
- If you want to create one list item to be un-clicked used. disable class

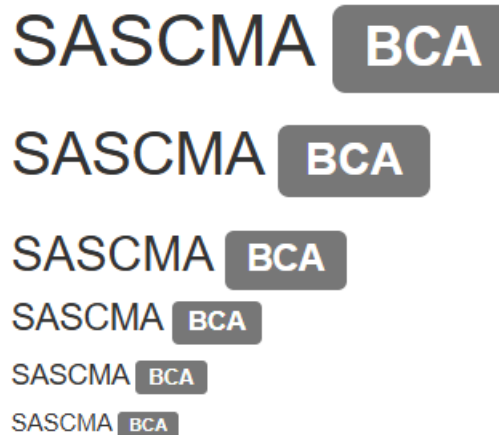
```
<li class="page-item disabled"><a class="page-link" href="#">Previous</a></li>
```

2.7 Bootstrap Grids

- Covered mention topic in first portion

2.8 Bootstrap Theme

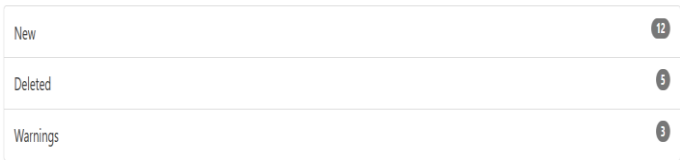
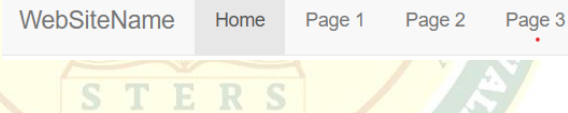
```
<div class="container">
  <h1>SASCMA <span
class="label label-
default">BCA</span></h1>
  <h2>SASCMA <span
class="label label-
default">BCA</span></h2>
  <h3>SASCMA <span
class="label label-
default">BCA</span></h3>
  <h4>SASCMA <span
class="label label-
default">BCA</span></h4>
  <h5>SASCMA <span
class="label label-
default">BCA</span></h5>
  <h6>SASCMA <span
class="label label-
default">BCA</span></h6>
</div>
```



```
<div class="progress">
  <div class="progress-bar"
role="progressbar" aria-
valuenow="70" aria-
```

Basic Progress Bar



<pre> valuemin="0" aria- valuemax="100" style="width:70%"> 70% Complete </div> </div> </pre>	
<pre> <ul class="list-group"> <li class="list-group- item">New 12 <li class="list-group- item">Deleted 5 <li class="list-group- item">Warnings 3 </pre>	
<pre> <ul class="dropdown-menu"> HTML CSS JavaScript About Us </pre>	
<pre> <nav class="navbar navbar- default"> <div class="container-fluid"> <div class="navbar-header"> WebSiteName </div> <ul class="nav navbar-nav"> <li class="active">Home Page 1 Page 2 Page 3 </div> </nav> </pre>	

- A **Bootstrap Theme** is a collection of styles that changes the appearance of a Bootstrap website **without changing its HTML structure**. Themes typically customize colors, fonts, buttons, cards, navigation bars, and other components to create a unique look.
[With a above all class and with the help of CSS, HTML create your own Bootstrap template which will be future used for project.]

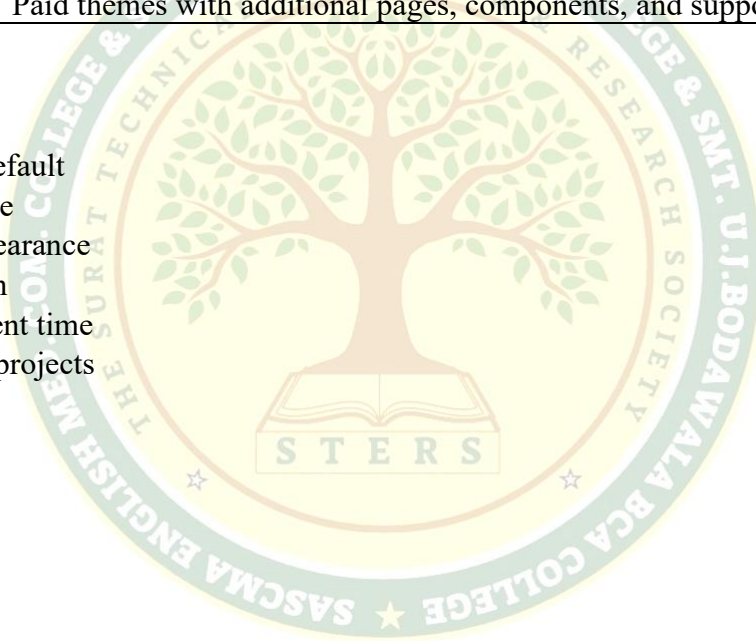
Why use a Bootstrap Theme?

- Gives your website a unique appearance.
- Maintains Bootstrap's responsive design.
- Saves development time.
- Provides a professional and consistent UI

Theme Type	Description
Default Theme	Standard Bootstrap look and feel.
Custom Theme	Created by changing Bootstrap colors, fonts, spacing, etc.
Free Theme	Freely available templates built with Bootstrap.
Premium Theme	Paid themes with additional pages, components, and support.

Advantages

- Responsive by default
- Easy to customize
- Professional appearance
- Consistent design
- Saves development time
- Reusable across projects





BOOTSTRAP 4 COMPLETE MANUAL FOR BEGINNERS

(WITH EXAMPLES & RESPONSIVE DESIGN)

Bootstrap

1. INTRODUCTION TO BOOTSTRAP

Bootstrap is the most popular HTML, CSS, JS framework for developing responsive, mobile first websites.

Features:

- ✓ Responsive
- ✓ Mobile First
- ✓ Easy to use
- ✓ Cross browser
- ✓ Re-usable components
- ✓ Based on HTML, CSS, JS

Version

Bootstrap 3 → 2013
Bootstrap 4 → 2018
Bootstrap 5 → 2021

Difference Between Versions

Feature	BS 3	BS 4	BS 5
Grid System	12 Column	12 Column	12 Column
jQuery Dependency	Yes	Yes	No
Font Awesome	3	4	5
Browser Support	Old	Better	Latest
Card Component	No	Yes	Yes

2. INSTALLING BOOTSTRAP 4

A. Using CDN (Recommended)

CSS

```
<link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css">
```

JS (jQuery, Popper.js, Bootstrap)

```
<script src="https://code.jquery.com/jquery-3.5.1.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/popper.js@1.16.0/dist/umd/popper.min.js"></script>
<script src="https://maxcdn.bootstrapcdn.com/bootstrap/4.5.2/js/bootstrap.min.js"></script>
```

B. Download Bootstrap

Download from getbootstrap.com → Extract → Include CSS and JS files.

C. Folder Structure

```
project/
├── css/
│   └── bootstrap.min.css
├── js/
│   ├── jquery.min.js
│   ├── popper.min.js
│   └── bootstrap.min.js
└── index.html
```

3. RESPONSIVE WEB DESIGN

Bootstrap makes websites look good on all devices.

Key Breakpoints

Extra Small	< 576px
Small	≥ 576px
Medium	≥ 768px
Large	≥ 992px
Extra Large	≥ 1200px

Example

Desktop → Tablet → Mobile



Breakpoints in Bootstrap 4

Device	Class Prefix	Screen Width
Extra Small (Phones)	.col-	< 576px
Small (Landscape Phones)	.col-sm-	≥ 576px
Medium (Tablets)	.col-md-	≥ 768px
Large (Desktops)	.col-lg-	≥ 992px
Extra Large (Big Desktops)	.col-xl-	≥ 1200px

4. GRID SYSTEM

- Bootstrap 4 uses 12 column grid system.
- Container → Rows → Columns
- Columns must be inside rows.

12 Columns Structure

1 2 3 4 5 6 7 8 9 10 11 12

Example : 3 Equal Columns

```
<div class="container">
  <div class="row">
    <div class="col-4 bg-primary p-3 text-white">Column 1</div>
    <div class="col-4 bg-success p-3 text-white">Column 2</div>
    <div class="col-4 bg-warning p-3 text-dark">Column 3</div>
  </div>
</div>
```

Column 1 Column 2 Column 3

5. TYPOGRAPHY

Bootstrap provides beautiful typography.

H1. Bootstrap Heading

H2. Bootstrap Heading

H3. Bootstrap Heading

H4. Bootstrap Heading

H5. Bootstrap Heading

H6. Bootstrap Heading

Display Headings

display-1 display-2 display-3 display-4

Lead Text

```
<p class="lead">This is lead paragraph.</p>
```

Text Alignment

```
<p class="text-left">Left</p>    <p class="text-center">Center</p>
<p class="text-right">Right</p>
<p class="text-justify">Justify</p>
<p class="text-normal">Normal</p>
```

Text Transformation

text-lowercase text-uppercase text-capitalize

Blockquote

```
<blockquote class="blockquote">
  <p>This is a blockquote</p>
  <div class="text-right">
    <p>— Someone's name</p>
  </div>
</blockquote>
```

6. COLORS

Text Colors

```
<p class="text-primary">Primary Text</p>
<p class="text-secondary">Secondary Text</p>
<p class="text-success">Success Text</p>
<p class="text-warning">Warning Text</p>
<p class="text-info">Info Text</p>
<p class="text-light">Light Text</p>
<p class="text-dark">Dark Text</p>
<p class="text-white">White Text</p>
```

Background Colors

```
<div class="bg-primary">Primary</div>
<div class="bg-success">Success</div>
<div class="bg-danger">Danger</div>
<div class="bg-warning">Warning</div>
<div class="bg-info">Info</div>
<div class="bg-dark">Dark</div>
<div class="bg-light">Light</div>
<div class="bg-white">White</div>
```

Example:

```
<div class="text-white bg-primary p-3">Primary</div>
<div class="text-white bg-success p-3">Success</div>
<div class="text-white bg-danger p-3">Danger</div>
```

7. SPACING (MARGIN & PADDING)

Spacing classes from 0 to 5

m = margin p = padding
t = top b = bottom l = left r = right
x = left + right y = top + bottom

Margin Example

m-0 m-1 m-2 m-3 m-4 m-5

Padding Example

p-0 p-1 p-2 p-3 p-4 p-5

Example:

```
<div class="m-3 bg-primary text-white p-3">Margin</div>
<div class="p-4 bg-success text-white p-4">Padding</div>
<div class="p-5 bg-warning text-dark p-5">Padding</div>
<div class="p-2 bg-info text-white p-2">Padding</div>
```

8. BORDERS

Border Classes

- border
- border-top
- border-right
- border-bottom
- border-left
- border-0 (no border)

Rounded Corners

- rounded
- rounded-top
- rounded-right
- rounded-bottom
- rounded-left
- rounded-circle
- rounded-pill
- rounded-0

Example:

```
<div class="border p-3 mb-2">Border Box</div>
<div class="border rounded p-3 mb-2">Rounded</div>
<div class="img-fluid" style="width:100px; height:100px;"></div>
```

9. IMAGES

Image Classes

- img-fluid → Responsive image
- rounded → Rounded image
- rounded-circle → Circle image
- img-thumbnail → Thumbnail image

Example:

```




```

10. TABLES

Table Classes

- table
- table-bordered
- table-striped
- table-hover
- table-dark
- table-sm (small table)
- table-responsive

Example

```
<table>
  <tr>
    <th>#</th>
    <th>Name</th>
    <th>Marks</th>
  </tr>
  <tr>
    <td>1</td>
    <td>Revi</td>
    <td>85</td>
  </tr>
  <tr>
    <td>2</td>
    <td>Ajay</td>
    <td>90</td>
  </tr>
</table>
```

11. ALERTS

Alert Classes

- alert-success
- alert-info
- alert-warning
- alert-danger

Example:

```
<div class="alert alert-success">This is success.</div>
<div class="alert alert-info">This is info.</div>
<div class="alert alert-warning">This is warning.</div>
<div class="alert alert-danger">This is danger.</div>
```

Dismissable Alert

```
<div class="alert alert-info alert-dismissible fade show">
  <button type="button" class="close"></button>
  <div>This is a dismissible alert.</div>
</div>
```

12. BUTTONS

Button Classes

- btn
- btn-primary
- btn-success
- btn-danger
- btn-warning
- btn-info
- btn-dark
- btn-light
- btn-link

Sizes

btn-lg btn-sm

Button Outline

```
<button class="btn btn-primary">Primary</button>
<button class="btn btn-outline-primary">Outline Primary</button>
```

Active Button

```
<button class="btn btn-primary active">Active Button</button>
```

13. BADGES

Badge Classes

- badge
- badge-primary
- badge-success
- badge-danger
- badge-warning
- badge-info
- badge-dark

Example

```
<span class="badge badge-primary">Primary</span>
<span class="badge badge-success">Success</span>
<span class="badge badge-danger">Danger</span>
<span class="badge badge-warning">Warning</span>
<span class="badge badge-info">Info</span>
<span class="badge badge-dark">Dark</span>
```

Pill Badge

```
<span class="badge badge-primary badge-pill">Pill</span>
```

Button with Badge

```
<button class="btn btn-primary">
  <span class="badge badge-light">4</span>
</button>
```

14. PROGRESS BAR

Example

```
<div class="progress">
  <div class="progress-bar bg-success" style="width:75%;></div>
</div>
<div class="progress">
  <div class="progress-bar bg-info" style="width:40%;></div>
</div>
<div class="progress">
  <div class="progress-bar bg-warning" style="width:60%;></div>
</div>
<div class="progress">
  <div class="progress-bar bg-danger" style="width:30%;></div>
</div>
```

Animated Progress Bar

```
<div class="progress">
  <div class="progress-bar progress-bar-striped progress-bar-animated" style="width:100%;></div>
</div>
```

19. CARDS

Card Example

```
<div class="card">
  <div class="card-body">
    <div class="img-fluid" style="width:100px; height:100px;"></div>
    <div class="card-title">Card Title</div>
    <p>Some quick example text to build on the card title and make up the bulk of the card's content.</p>
    <button class="btn btn-primary">Read More</button>
  </div>
</div>
```

16. NAVIGATION BAR

Navbar Example

```
<div class="navbar">
  <div class="container">
    <div class="brand">Brand</div>
    <div class="list">
      <a href="#">Home</a>
      <a href="#">About</a>
      <a href="#">Services</a>
      <a href="#">Contact</a>
    </div>
  </div>
</div>
```

17. FORMS

Form Example

```
<div class="form">
  <div class="form-group">
    <input type="text" value="Enter your name">
  </div>
  <div class="form-group">
    <input type="email" value="Enter email">
  </div>
  <div class="form-group">
    <div>
      <input type="radio"/> Male
      <input checked="" type="radio"/> Female
    </div>
  </div>
  <div class="form-group">
    <div>
      <input type="checkbox"/> Reading
      <input type="checkbox"/> Music
      <input type="checkbox"/> Sports
    </div>
  </div>
  <div class="form-group">
    <div>
      <div>Select Country</div>
      <div>India</div>
    </div>
  </div>
  <div class="form-group">
    <div>
      <button type="submit" value="Submit">Submit</button>
      <button type="reset" value="Reset">Reset</button>
    </div>
  </div>
</div>
```

20. IMAGE CLASSES

Classes

- Accordion
- Collapse
- rounded-circle
- img-thumbnail

Image Alignment

- float-left
- float-right
- img-auto
- block
- center

Example

```



```

21. UTILITIES

Text Utilities

- text-muted
- text-bold
- text-italic
- text-underline
- text-decoration
- text-decoration
- text-decoration

Flexbox

- justify-content: center
- justify-content: space-between
- align-items: center
- justify-content: space-around
- align-items: flex-end

15. MEDIA OBJECT

Media Object Example

```
<div class="media">
  <div class="media-object">
    
  </div>
  <div class="media-body">
    <p>This is an example of Bootstrap media object. Used for comments, blog posts, user profiles, etc.</p>
  </div>
</div>
```

Nested Media Object

```
<div class="media">
  <div class="media-object">
    
  </div>
  <div class="media-body">
    <div class="media">
      <div class="media-object">
        
      </div>
      <div class="media-body">
        <p>This is a nested Bootstrap media object.</p>
      </div>
    </div>
  </div>
</div>
```

21. PAGINATION

Pagination Example

```
<div class="pagination">
  <span>Previous</span>
  <span>1</span>
  <span>2</span>
  <span>3</span>
  <span>Next</span>
</div>
```

Disabled Pagination

```
<div class="pagination">
  <span class="disabled">Previous</span>
  <span>1</span>
  <span>2</span>
  <span>3</span>
  <span>Next</span>
</div>
```

Large Pagination

```
<div class="pagination">
  <span>Previous</span>
  <span>1</span>
  <span>2</span>
  <span>3</span>
  <span>Next</span>
</div>
```

Small Pagination

```
<div class="pagination">
  <span>Previous</span>
  <span>1</span>
  <span>2</span>
  <span>3</span>
  <span>Next</span>
</div>
```

21. COMPONENTS

Accordion

- Accordion
- Collapse
- Modal
- Popover
- Breadcrumb
- Pagination
- List Group
- Media Object
- Toast

Modal Example

```
<div class="modal">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">
        <div>Modal Title</div>
        <button type="button" class="close"></button>
      </div>
      <div class="modal-body">
        <p>This is modal body.</p>
      </div>
      <div class="modal-footer">
        <button type="button" value="Close">Close</button>
        <button type="button" value="Save changes">Save changes</button>
      </div>
    </div>
  </div>
</div>
```

23. BOOTSTRAP THEMES

Bootstrap Theme changes the appearance of your website.

Use Bootswatch (Free Themes)

- Cerulean
- Flatly
- Journal
- Paper
- Sandstone
- Simplex
- Spacelab
- Superhero
- United
- Yeti

How to Use

```
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/bootswatch/4.2.0/dark/bootstrap.min.css">
```

24. COMPLETE PROJECTS (IDEAS)

1. College Website

25. PRACTICAL EXERCISES (40+)

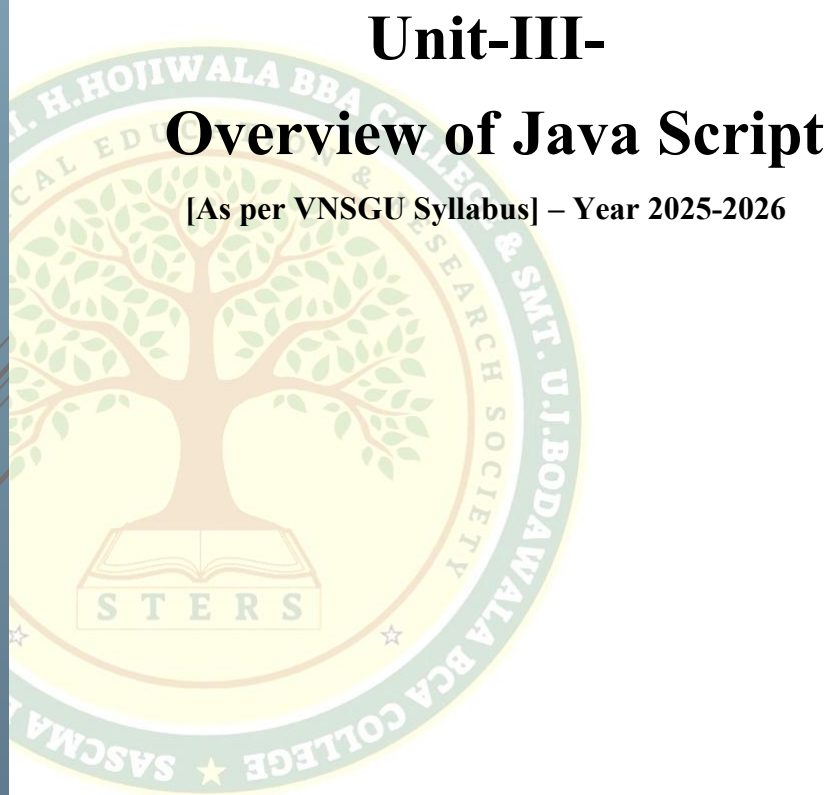
1. Create a responsive navigation bar
2. Create 3 column layout using card

26. VIVA QUESTIONS (100+)

1. What is Bootstrap?
2. What is grid system?

Unit-III-
Overview of
Java Script

Unit – V
JavaScript
Functions



Index

Content

- 3.1 Overview of client & Server-Side Scripting
- 3.2 Structure of Java Script
- 3.3 Data types and Variable
- 3.4 Operators (Arithmetic. Assignment, Comparison)
Logical and Conditional Operator)
- 3.5 Control Structure
- 3.5.1 If. . Else, switch. Case
- 3.5.2 While, Do. . While, For Loop
- 3.5.3 break, continue
- 3.6 JavaScript String and Events
- 3.6.1 Javascript Strings types
- 3.6.2 String functions: concat(), split(), indexOf(), lastIndexOf().
substring(), trim(), slice(), replace(), charAt()
- 3.6.3 Javascript Events :
- 3.6.3.1 Mouse Events : (click, mouseover, mousemove, mouseout,
mouseup)
- 3.6.3.2 keyboard Events : (keyup, keydown)
- 3.6.3.3 Form Event : (focus, submit, blur, change)
- 5.1 JavaScript Functions
- 5.2 Dialog boxes : Alert, confirm, prompt
- 5.3 Form validation

3.1 Overview of client & Server-Side Scripting

A **script** is a set of instructions executed by a browser or server. A script is a short set of instructions used to automate tasks, while JavaScript is a powerful, versatile programming language that makes websites interactive and dynamic.

- It is interpreted and executed line-by-line at runtime, rather than requiring full ahead-of-time compilation.

Types of Scripting

1. Server-side scripting
2. Client-side scripting

Client-side scripting	Server-side scripting
Source code is visible to the user.	Source code is not visible to the user because output of server-side is an HTML page.
Its main function is to provide the requested output to the end user.	Its primary function is to manipulate and provide access to the database as per the request.
It usually depends on the browser and its version.	In this any server-side technology can be used and it does not depend on the client.
It runs on the user's computer.	It runs on the webserver.
There are many advantages linked with this like faster. response times, a more interactive application.	The primary advantage is its ability to highly customize, response requirements, access rights based on user.
It does not provide security for data.	It provides more security for data.
It is a technique used in web development in which scripts run on the client's browser.	It is a technique that uses scripts on the webserver to produce a response that is customized for each client's request.
HTML, CSS, and javascript are used.	PHP, Python, Java, Ruby are used.
No need of interaction with the server.	It is all about interacting with the servers.
It reduces load on processing unit of the server.	It surges the processing load on the server.

What is JavaScript?

- **Definition:** JavaScript (JS) is a lightweight, high-level, interpreted programming language of the web.
- It was originally named "LiveScript" but was renamed to take advantage of Java's popularity at the time. Alongside HTML (structure) and CSS (styling), JavaScript is the foundational layer responsible for web page behavior.
- It can update and change both HTML and CSS. It can calculate, manipulate and validate data.
- JavaScript is easy to learn. Which don't have to get or download JavaScript.
- While HTML and CSS create static pages, JavaScript brings them to life. It runs directly inside the user's web browser without needing to compile beforehand.

3.2 Structure of JavaScript

Where we insert JavaScript?

- **Internal JavaScript**
 - In HTML, JavaScript code is inserted between `<script>` and `</script>` tags which can be placed in both head or body tag. You can place any number of scripts in an HTML document.
- **External JavaScript**
 - JavaScript programs can also be stored in **external files** with the **.js extension**. An external JavaScript file can be linked to multiple web pages, making the code **reusable, easier to maintain, and more organized**. External scripts cannot contain `<script>` tags. Cached JavaScript files can speed up page loads.

Inline JavaScript	Internal JavaScript	External JavaScript
<code><button onclick="alert('Hello')"> Click</button></code>	<code><script> document.write(" Welcome"); </script></code>	<code><script src="script.js"></script></code>
Easy for small tasks.	Need to write script tag	Reusable
No separate script section required.	May use script tag in body or head tag.	Faster page loading (browser caching)

Difficult to maintain	Easy for single page	Easy to maintain
Not suitable for large applications.	Not suitable for large applications.	Cleaner HTML code as script and HTML code is separate
		An external script can be referenced in 3 different ways: -With a full URL (a full web address) -With a file path (like /js/) -Without any path
		<pre><script src="https://www.w3schools.com/js/myScript.js"></script> <script src="/js/myScript.js"></script> <script src="myScript.js"></script></pre>

JavaScript Comments

Symbol	Description
//	Single line comment
/* */	Multi-line comment

Display contains on webpage

document.write() - is a JavaScript method used to **display text, numbers, HTML tags, and variable values directly on the web page** while the page is loading.

```
<script> document.write("Welcome to JavaScript"); </script>
```

```
<script> document.write(100); </script>
```

```
<script> document.write("<h2>Welcome Students</h2>"); </script>
```

```
<script> document.write(new Date()); </script>
```

```
document.write("<br>");
```

Advantage	Disadvantage
Simple for beginners. - Useful for demonstrations and small examples.	Intended mainly for learning and debugging.
Can display variables and HTML.	If called after the page has fully loaded, it can overwrite the existing page content.

document.getElementById() is a JavaScript method used to **find and access an HTML element using its unique id attribute**. Once the element is found, you can change its content, style, or other properties. Document represent current HTML page. `getElementById()` searches for an element by its id.

<pre> <!DOCTYPE html> <html> <body> <h2 id="demo">Hello Students</h2> <button onclick="changeText()"> Click Here </button> <script> function changeText() { document.getElementById("demo").innerHT ML="Welcome to JavaScript"; } </script> </body> </html> </pre>	<pre> <p id="text">JavaScript Program change text</p> <button onclick="changeColor()"> Change Color </button> <script> function changeColor() { document.getElementById("text").style. color="red"; } </script> </pre>
--	--

Various properties used with getElementById()

Property	Purpose	Example
innerHTML	Changes HTML content	element.innerHTML = "Hello";
innerText	Changes plain text	element.innerText = "Welcome";
value	Gets or sets form input values	input.value
style.color	Changes text color	element.style.color = "blue";
style.backgroundColor	Changes background color	element.style.backgroundColor = "yellow";

src	Changes image source	image.src = "photo.jpg";
document.write()		document.getElementById()
Purpose is to writes content to the page while it is loading		Purpose is to accesses an existing HTML element
Requires HTML element to display content		HTML element ID is required to perform task
Can't modify existing content		Can modify existing content
Can't use with events		It can be used with events
Not Suitable for interactive pages		Suitable for interactive pages

Old JavaScript examples may use a type attribute: `<script type="text/javascript">`.

The type attribute is not required. JavaScript is the default scripting language in HTML.

3.3 Data types and Variable

JavaScript variables are containers for data. JavaScript variables can be **declared** in 4 ways:

Modern JavaScript

- Using let
- Using const
- Using var
- Automatically

Variables are identified with **names** called **identifiers**.

Rules for Identifier:

1. Names can **contain** letters, digits, underscores, and dollar signs.
2. Names must **begin** with a letter, a \$ sign or an underscore (_).
3. Names are **case sensitive** (X is different from x).
4. **Reserved words** (JavaScript keywords) cannot be used as names.
5. Numbers are not allowed as the first character in names.

Variable	Constants
1. With the use of let keyword	

<pre><script> let x = 5; let y = 6; let z = x + y; document.getElementById("demo").innerHTML = "The value of z is " + z; </script></pre> - Variables declared with let have Block Scope - Variables declared with let must be Declared before use - Variables declared with let cannot be Redeclared in the same scope	With the use of const keyword <pre><script> const x = 5; const y = 6; const z = x + y; document.getElementById("demo").innerHTML = "The value of z is " + z; </script></pre>
2. With the use of var keyword Variables declared with the var always have Global Scope. <pre>{ var x = 2; // var can be redeclared } // x CAN be used here</pre>	

JavaScript Datatype

Type	Description
String	A text of characters enclosed in quotes[Use single quote or double quote] <pre><p id="demo"></p> <script> const pi = 3.14; // constant let a = "Value of "; // String variable</pre>

	<pre> let b = 'PI'; document.getElementById("demo").innerHTML = pi + "
" + a + "
" + b; let x=2 + 3 + "5"; document.getElementById("demo").innerHTML = x; //output will be 55 [2+3 = 5 and last 5 will be worked as string] </script> </pre>
Number	A number representing a mathematical value <pre> <p id="demo"></p> <script> let x1 = 34.00; let x2 = 34; let x3 = 3.14; let y = 123e5; // 12300000 let z = 123e-5; // 0.00123 document.getElementById("demo").innerHTML =x1 + "
" + x2 + "
" + x3 + "
" + y + "
" + z; </script> </pre>
Bigint	A number representing a large integer
Boolean	A data type representing true or false <pre> <p id="demo"></p> <script> let x = 5; let y = 5; </pre>

	<pre>let z = 6; document.getElementById("demo").innerHTML =(x == y) + "
" + (x == z); </script></pre>
Object	A collection of key-value pairs of data JavaScript objects are written with curly braces {}. Object properties are written as name:value pairs, separated by commas <pre><p id="demo"></p> <script> const person = { firstName : "John", lastName : "Doe", age : 50, eyeColor : "blue" }; document.getElementById("demo").innerHTML =person.firstName + " is " + person.age + " years old."; </script></pre>
Undefined	A primitive variable with no assigned value <pre><p id="demo"></p> <script> let car; document.getElementById("demo").innerHTML = car + "
" + typeof car;</pre>

	<pre> </script> <p id="demo1"></p> <script> let car1 = "Volvo"; car1 = undefined; document.getElementById("demo1").innerHTML = car1 + "
" + typeof car1; </script> </pre>
Null	A primitive value representing object absence
Symbol	A unique and primitive identifier

JavaScript Array

- JavaScript arrays are written with square brackets.
- Array items are separated by commas.

```

<p id="demo"></p>
<script>
const cars = ["BCA","BBA","B.com"];
document.getElementById("demo").innerHTML = cars[0];
</script>

```

3.4 Operators (Arithmetic, Assignment, Comparison, Logical and Conditional Operator)

Arithmetic Operators

Operator	Example
Arithmetic Operators + , - , * , / , ++ , --	<pre> <p id="demo2"></p> <script> let d = 5; let e = 2; let f = x * y; document.getElementById("demo2").innerHTML = f; </script> </pre>
	<pre> <!DOCTYPE html> <html> </pre>

	<pre> <head> <title>Calculator</title> </head> <body> <script> let a = Number(prompt("Enter First Number")); let b = Number(prompt("Enter Second Number")); document.write("Addition = " + (a+b) + "
"); document.write("Subtraction = " + (a-b) + "
"); document.write("Multiplication = " + (a*b) + "
"); document.write("Division = " + (a/b)); </script> </body> </html> </pre>
Assignment Operators =, +=, -=, *=, /=, %=, **=,	<pre> <p id="demo3"></p> <script> let a = 10; a *= 5; document.getElementById("demo3").innerHTML = a; </script> </pre>
Comparison Operators ==, ===, !=, !==, >, <, >=, <=	<pre> <p id="demo1"></p> <script> let y = 5; document.getElementById("demo1").innerHTML = (y === 5); </script> </pre>
	<pre> <!DOCTYPE html> <html> <head> <title>Largest Number</title> </head> <body> <script> let a = Number(prompt("Enter First Number")); let b = Number(prompt("Enter Second Number")); if(a>b) { document.write(a + " is Largest"); } else { document.write(b + " is Largest"); } </pre>

	<pre> } </script> </body> </html> </pre>
Logical Operators &&, ,!	<pre> <p id="demo"></p> <script> let x = 6; let y = 3; document.getElementById("demo").innerHTML = (x < 10 && x > 1) + "
" + (y < 10 && y < 1); </script> </pre>
Conditional Operators	<pre> variablename = (condition) ? value1:value2 let voteable = (age < 18) ? "Too young":"Old enough"; document.getElementById("demo").innerHTML = voteable + " to vote."; </pre>

3.5 Control Structure

Control Structure	Example
If Uppercase letters (If or IF) will generate a JavaScript error.	<pre> <p id="demo">Good Evening!</p> <script> if (new Date().getHours() < 18) { document.getElementById("demo").innerHTML = "Good day!"; } / / it will check and if time less than 18(6:00 PM than display good day) </script> </pre>
If else	<pre> <p id="demo"></p> <script> const hour = new Date().getHours(); let greeting; if (hour < 18) { greeting = "Good day"; } else { greeting = "Good evening"; } </pre>

	<pre>document.getElementById("demo").innerHTML = greeting; </script></pre>
Nested if	<pre><p id="demo"></p> <script> let age = 16; let country = "INDIA"; let text = "You can Not drive!"; if (country == "INDIA") { if (age >= 16) { text = "You can drive!"; } } if (country == "USA" && age >= 16) { text = "You can drive!"; } document.getElementById("demo").innerHTML = text; </script></pre>
Switch	<pre>switch (new Date().getDay()) { case 6: text = "Today is Saturday"; break; case 0: text = "Today is Sunday"; break; default: text = "Looking forward to the Weekend"; }</pre>
While	<pre><p id="demo"></p> <script> let text = ""; let i = 0; while (i < 10) { text += "
The number is " + i; i++; } document.getElementById("demo").innerHTML = text; </script></pre>

Do while	<pre> <p id="demo"></p> <script> let text = "" let i = 0; do { text += "
The number is " + i; i++; } while (i < 10); document.getElementById("demo").innerHTML = text; </script> </pre>
For	<pre> <p id="demo"></p> <script> let text = ""; for (let i = 0; i < 5; i++) { text += "The number is " + i + "
"; } document.getElementById("demo").innerHTML = text; </script> </pre>
	<pre> <script> let n = Number(prompt("Enter Number")); for(let i=1;i<=10;i++) { document.write(n + " × " + i + " = " + (n*i) + "
"); } </script> </pre>
Break	<pre> <p id="demo"></p> <script> let text = ""; for (let i = 0; i < 10; i++) { if (i === 3) { break; } text += "The number is " + i + "
"; } document.getElementById("demo").innerHTML = text; </pre>

	</script>
Continue	<p id="demo"></p> <script> let text = ""; for (let i = 0; i < 10; i++) { if (i === 3) { continue; } text += "The number is " + i + " "; } document.getElementById("demo").innerHTML = text; </script>

3.6 JavaScript String and Events

- A JavaScript string is zero or more characters written inside quotes.
- Strings created with single or double quotes work the same

```
let x="Hello from BCA";
let y="Hello I am sascma's student";
let text= 'It's alright.';
```

String Functions

Count the length of string text.length	<p id="demo"></p> <script> let text = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"; document.getElementById("demo").innerHTML = text.length; </script>
charAt() charAt() method returns the character at a specified index (position) in a string	let text = "HELLO WORLD"; let char = text.charAt(0); //Answer will be H
charCodeAt() method returns the code of the character at a specified index in a string	<p id="demo"></p> <script> let text = "HELLO WORLD"; document.getElementById("demo").innerHTML = text.charCodeAt(0); </script>

he method returns a UTF-16 code (an integer between 0 and 65535).	
concat() joins two or more strings, method can be used instead of the plus operator. Strings are immutable: Strings cannot be changed, only replaced. [All string methods return a new string. They don't modify the original string]	<pre><p id="demo"></p> <script> let text1 = "Hello"; let text2 = "World!"; let text3 = text1.concat(" ",text2); document.getElementById("demo").innerHTML = text3; </script></pre>
Split() A string can be converted to an array with the split() method. split("") is always unsafe for emojis. It will slice right through them and create broken, corrupt code fragments.	<pre><p id="demo"></p> <script> let text = "Hi BCA Students!"; const myArr = text.split(""); document.getElementById("demo").innerHTML = JSON.stringify(myArr); </script></pre>
indexOf() method returns the position of the first occurrence of a value in a string. method returns -1 if the value is not found.	<pre><p id="demo2"></p> <script> let text2 = "Hello world, welcome to the universe."; document.getElementById("demo2").innerHTML = text2.indexOf("e"); </script></pre>
substring() substring() method extracts characters from start to end (exclusive).	<pre><p id="demo"></p> <script> let text = "Hello world!"; let result = text.substring(1,7); document.getElementById("demo").innerHTML = result;</pre>

string.substring(<i>start</i>, <i>end</i>) If start is greater than end, arguments are swapped: (4, 1) = (1, 4).	<pre></script></pre>
trim() method removes whitespace from both sides of a string.	<pre><pre><p id="demo1"></p></pre> <pre><p id="demo2"></p></pre> <script> let text = " Hello World!"; let result = text.trim(); document.getElementById("demo1").innerHTML = text; document.getElementById("demo2").innerHTML = result; </script></pre>
slice() method extracts a part of a string. method returns the extracted part in a new string	<pre><p id="demo"></p> <script> let text = "Hello world!"; let result = text.slice(0, 7); document.getElementById("demo").innerHTML = result; </script></pre>
replace() method searches a string for a value or a regular expression. method returns a new string with the value(s) replaced string.replace(searchValue, newValue)	<pre><p id="demo">Visit Microsoft!</p> <script> let text = document.getElementById("demo").innerHTML; document.getElementById("demo").innerHTML = text.replace("Microsoft", "Infosys"); </script></pre>
lastIndexOf() method returns the index (position) of the last occurrence of a specified value in a string.	<pre><p id="demo"></p> <script> let text = "Hello planet earth, you are a great planet."; let result = text.lastIndexOf("planet"); document.getElementById("demo").innerHTML = result; </script></pre>

method searches the string
from the end to the
beginning.

Escape Character

- Because strings must be written within quotes, JavaScript will misunderstand this string.
- `let text = "We are the so-called "Vikings" from the north.";`
- The string will be chopped to "We are the so-called ".
- The solution to avoid this problem is to use the backslash escape character.
- The backslash (\) escape character turns special characters into string characters:

Code	Result
<code>\'</code>	'
<code>\"</code>	"
<code>\\</code>	\
<code>\b</code>	Back space
<code>\f</code>	Form feed
<code>\n</code>	New Line
<code>\r</code>	Carriage Return
<code>\t</code>	Horizontal tabulator
<code>\v</code>	Vertical tabulator

```
<p id="demo1"></p>
<script>
let text1 = 'It\'s alright.';
document.getElementById("demo1").innerHTML = text1;
</script>
<p id="demo2"></p>
<script>
let text2 = "The character \\ is called backslash.";
document.getElementById("demo2").innerHTML = text2;
</script>
```

Defining function (with and without parameters)

- A JavaScript function is a block of code designed to perform a particular task.
- A JavaScript function is executed when "something" invokes it (calls it).

<pre>function myFunction(p1, p2) { return p1 * p2; // The function returns the product of p1 and p2 }</pre>	<pre>function functionname(parameter1, parameter2, parameter3) { // code to be executed }</pre>
---	---

- Inside the function, the arguments (the parameters) behave as local variables.
- let x = myFunction(4, 3); // Function is called, return value will end up in x
function myFunction(a, b) {
return a * b; // Function returns the product of a and b }
- The code inside the function will execute when "something" **invokes** (calls) the function:
 - a) When an event occurs (when a user clicks a button)
 - b) When it is invoked (called) from JavaScript code
 - c) Automatically (self invoked)

JavaScript Events

A JavaScript Event is an action or occurrence that happens in the browser. JavaScript can detect these actions and execute a function (event handler) in response.

Example of events: Clicking a button, typing in a textbox, Loading a webpage, Moving the mouse, pressing a keyboard key, submitting a form, Resizing the browser window

With single quotes:

<element event='some JavaScript'>

With double quotes:

<element event="some JavaScript">

Event	Description	
onclick		Single click
Button Click	Occurs when user clicks an element	

Ondblclick	Double-click on an element	Double Click
onmouseover	Mouse moves over an element	Hover
onmouseout	Mouse leaves an element	Mouse Out
onmousedown	Mouse button is pressed	Mouse Down
onmouseup	Mouse button is released	Mouse Up
onmousemove	Mouse moves over an element	Mouse Tracking
onkeydown	Key is pressed	Keyboard Input
onkeyup	Key is released	Search Box
onkeypress	Key is typed (<i>deprecated</i>)	
onfocus	Element gets focus	Click inside textbox
onblur	Element loses focus	Leave textbox
onchange	Value of input changes	Select Box
oninput	Runs whenever value changes	Live Typing
onsubmit	Form is submitted	Form Validation
onreset	Form is reset	Reset Button
onload	Page or image finishes loading	Welcome Message
onunload	Page is closed (<i>deprecated for many uses</i>)	Cleanup
onresize	Browser window is resized	Responsive Layout
onscroll	User scrolls the page	Infinite Scroll

Examples:

Change background color when click on button	Read value from textbox
<pre><button onclick="changeBackground()"> Background </button> <script> function changeBackground() { document.body.style.background="yellow"; } </script></pre>	<pre><input type="text" id="name"> <button onclick="showName()"> Show </button> <p id="result"></p> <script> function showName() { let n=document.getElementById("name"). value;</pre>

	<pre>document.getElementById("result").innerHTML=n; } </script></pre>
Change text	Change textcolor
<pre><html> <body> <h2 id="demo">Hello Students</h2> <button onclick="changeText()"> Click Here </button> <script> function changeText() { document.getElementById("demo").innerHTML="Welcome to JavaScript"; } </script> </body> </html></pre>	<pre><p id="text">JavaScript Programming</p> <button ondblclick="changeColor()"> Change Color </button> <script> function changeColor() { document.getElementById("text").style .color="red"; } </script></pre>

JavaScript Event Handlers

An event handler is JavaScript code that runs when an event happens.

Event handlers can be used to handle and verify user input, user actions, and browser actions:

- Things that should be done every time a page load
- Things that should be done when the page is closed
- Action that should be performed when a user clicks a button
- Content that should be verified when a user inputs data

```
<button onclick="displayDate()">Time is?</button>
<p id="demo"></p>
<script>
// Function to display date
function displayDate() {
  document.getElementById("demo").innerHTML = Date();
}
</script>
```

Using an Event Listener

- The `addEventListener()` method keeps HTML and JavaScript separated.

```
<button id="myBtn">Click me</button>
<p id="demo"></p>
<script>
const btn = document.getElementById("myBtn");
// Add EventListener to btn
btn.addEventListener("click", function () {
  document.getElementById("demo").innerHTML = Date();
});
</script>
```

- Mouse Event:** Mouse events are events that occur when a user interacts with a webpage using a mouse. These events can be used to perform actions like clicking a button, hovering over an element, dragging objects, and more.

Mouse Event	Description
Mousedown	Triggered when a mouse button is pressed down. <pre><p id="myP" onmousedown="mouseDown()" onmouseup="mouseUp()">Click the text! </p> <script> function mouseDown() { document.getElementById("myP").style.color = "red"; } function mouseUp() { document.getElementById("myP").style.color = "green"; } </script></pre>
Mouseup	Triggered when a pressed mouse button is released.
Mouseenter	Triggered when the mouse pointer enters an element (does not bubble).
Mouseleave	Triggered when the mouse pointer leaves an element (does not bubble). <pre><div id="myBox" style="width: 200px; height: 200px; background-color: lightblue;"> Hover over me!</pre>

	<pre> </div> <script> const box = document.getElementById("myBox"); // Triggered when mouse leaves the box box.addEventListener("mouseleave", () => { box.style.backgroundColor = "lightblue"; }); box.addEventListener("mouseenter", () => { box.style.backgroundColor = "orange"; }); </script> </pre>
Mouseover	Triggered when the mouse pointer moves over an element or its child elements. <pre> <div id="box" style="width:200px;height:100px;background:lightblue;"> Hover over me</div> <script> const box = document.getElementById("box"); box.addEventListener("mouseover", function() { box.style.backgroundColor = "yellow";}); box.addEventListener("mouseout", function() { box.style.backgroundColor = "lightblue";}); </script> </pre>

- **Keyboard Event:**

onkeyup – Event occurs when the user releases a key on the keyboard.	<pre> <input type="text" onkeyup="alert('Key released!')"> </pre> <p>A function is triggered when the user releases a key in the input field.</p>
--	--

	<p>The function transforms the input field to upper case:</p> Enter your name: <input type="text" id="fname" onkeyup="myFunction()"> <pre> <script> function myFunction() { let x = document.getElementById("fname"); x.value = x.value.toUpperCase(); } </script> </pre>
onkeydown – event occurs when the user presses a key on the keyboard.	<p>Press a key in the input field:</p> <pre> <input type="text" onkeydown="myFunction()"> <p id="demo"></p> <script> function myFunction() { document.getElementById("demo").innerHTML = "You pressed a key inside the input field"; }</script> </pre> <u>Display Key Name which is pressed –</u> <pre> <input type="text" id="myInput" placeholder="Type here..."> <p>Key: </p> <script> </pre>

	<pre>const inputElement = document.getElementById('myInput'); inputElement.addEventListener('keydown', (event) => { document.getElementById('output').innerText = event.key; }); </script></pre>
onkeypress - event occurs when the user presses a key on the keyboard.	<pre><input type="text" onkeypress="myFunction()"> <script> function myFunction() { alert("You pressed a key inside the input field"); } </script></pre>

- **Form Event:**

onfocus:	<pre><input type="text" onfocus="makeYellow(this)" placeholder="Click or Tab here"> <script> function makeYellow(element) { element.style.background = "yellow"; } </script></pre> <pre><input type="text" id="myInput" placeholder="Focus me"></pre>
----------	--

	<pre> <script> const inputField = document.getElementById("myInput"); inputField.addEventListener("focus", (event) => { event.target.style.background = "lightgreen"; }); </script> </pre>
onsubmit	<pre> <!DOCTYPE html> <html> <body> <form action="#" onsubmit="myFunction()"> Enter name: <input type="text" name="fname"> <input type="submit" value="Submit"> </form> <script> function myFunction() { alert("The form was submitted"); } </script> </body> </html> </pre>
onblur	<pre> <input type="text" id="fname" onblur="myFunction()"> <script> </pre>

	<pre>function myFunction() { let x = document.getElementById("fname"); x.value = x.value.toUpperCase(); } </script></pre>
onchange	<pre><select id="mySelect" onchange="myFunction()"> <option value="Audi">Audi</option> <option value="BMW">BMW</option> <option value="Mercedes">Mercedes</option> <option value="Volvo">Volvo</option> </select> <p id="demo"></p> <script> function myFunction() { var x = document.getElementById("mySelect").value; document.getElementById("demo").innerHTML = "You selected: " + x;} </script></pre>

Redirect a Webpage

- There are a couple of ways to redirect to another webpage with JavaScript.
- The most popular ones are `location.href` and `location.replace`:

// Simulate a mouse click - `window.location.href = "http://www.w3schools.com";`

// Simulate an HTTP redirect -

```
window.location.replace("http://www.w3schools.com")
```

- The difference between href and replace, is that replace() removes the URL of the current document from the document history, meaning that it is not possible to use the "back" button to navigate back to the original document.

```
<script>
function myFunction() {
location.replace("https://www.w3schools.com")
}
</script>
```

5.2 Dialog Box

JavaScript provides **three native dialog boxes** through the window object. They are synchronous, modal, and block the execution of code until closed.

Dialog Box Type	Description & Example
Alert Box	An alert box is often used if you want to make sure information comes through to the user. When an alert box pops up, the user will have to click "OK" to proceed. <pre>window.alert("sometext");</pre> The window.alert() method can be written without the window prefix. Example: <pre><html> <body> <h2>JavaScript Alert</h2> <button onclick="myFunction()">Try it</button> <script> function myFunction() { alert("I am an alert box!"); } </pre>

	<pre> </script> </body> </html> </pre>
Confirm Box	A confirm box is often used if you want the user to verify or accept something. When a confirm box pops up, the user will have to click either "OK" or "Cancel" to proceed. If the user clicks "OK", the box returns true. If the user clicks "Cancel", the box returns false. <pre> window.confirm("sometext"); </pre> The window.confirm() method can be written without the window prefix. Example: <pre> <!DOCTYPE html> <html> <body> <h2>JavaScript Confirm Box</h2> <button onclick="myFunction()">Try it</button> <p id="demo"></p> <script> function myFunction() { var txt; if (confirm("Press a button!")) { txt = "You pressed OK!"; </pre>

```

} else {

txt = "You pressed Cancel!";

}document.getElementById("demo").innerHTML = txt;}

</script></body></html>

let isDelete = confirm("Are you sure you want to delete this
item?");

if (isDelete)

{

console.log("Item deleted."); // User clicked OK }

else { console.log("Action cancelled."); // User clicked Cancel

}

```

Prompt Box

A prompt box is often used if you want the user to input a value before entering a page.

When a prompt box pops up, the user will have to click either "OK" or "Cancel" to proceed after entering an input value.

If the user clicks "OK" the box returns the input value. If the user clicks "Cancel" the box returns null.

```
window.prompt("sometext","defaultText");
```

The window.prompt() method can be written without the window prefix

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```



```

<h2>JavaScript Prompt</h2>

<button onclick="myFunction()">Try it</button>

<p id="demo"></p>

<script>

function myFunction() {

let text;

let person = prompt("Please enter your name:", "Harry Potter");

if (person == null || person == "") {

text = "User cancelled the prompt.";

} else {

text = "Hello " + person + "! How are you today?"; }

document.getElementById("demo").innerHTML = text; }

</script>

</body>

</html>

```

- **JavaScript Form Validation**

JavaScript validation is used to check whether the data entered by a user in an HTML form is correct before submitting the form

Validation	Example
Required field	Name cannot be empty
Length	Password must contain at least 8 characters
Number	Age must contain only numbers
Email	Check valid email format

Password	Password and confirm password must match
Radio button	Gender must be selected
Checkbox	Terms & conditions must be accepted

Used to access an HTML element.

```
var name = document.getElementById("name").value;
```

Basic Validation

- Username and Password both required

```
<html>
<body>
<script language="javascript">
function form_validation()
{
name=document.form1.uname.value;
password=document.form1.pwd.value;
if(name=="")
{
alert("name is required");
return false;
}
if(password=="")
{
alert("please enter password");
return false;
}
}
</script>
<form name="form1" method="post" onsubmit="form_validation();">
student name:<input type="text" name="uname">
<br>
password:<input type="password" name="pwd">
<br>
<input type="submit" value="submit">
</form> </body></html>
```

- Data Format Validation

Sometimes it is necessary to check data for correct format and value. For example, an email address must enter in the proper format, the value must be a number, etc. This type of validation needs appropriate logic to test correctness of data.

a) Number validation -

- JavaScript can be used to validate input for numeric value only.
 - The isNaN() (Not-A-Number) function is used for this purpose.
- if(isNaN(age)) – Like this have to check for filed age

```
<html>

<body>

<script>

function num_validation() {

no = document.getElementById("no").value;

if (isNaN(no))

{ alert( "Input is not number");

}

else

{ alert( "Input is number");

} } </script>

<input type="number" id="no" >

<Button type="button" onclick="num_validation()">Submit</button>

</body>

</html>
```

b) Email validation:

- JavaScript can validate the email format.
- Email address follows various criteria such as:
- Must contain the @ and . character
- Minimum one character before and after the @
- Minimum two characters after . (dot).

Example –

```
<html>
<body>
<script type = "text/javascript">
function email_validation()
{
//alert("Please enter correct email ID");
Email_id = document.getElementById("email").value;
at_position = Email_id.indexOf("@");
dot_position = Email_id.lastIndexOf(".");
if((at_position<1)||((dot_position-at_position)<2))
{
alert("Please enter correct email ID");
return false;
}
else
{
alert("correct");
return true ;
}
}
</script>
<input id="email">
<button type="button" onclick="email_validation()">Submit</button>
</body>
</html>
```

c) Mobile no. validation:

```
<html>
<body>
<script>
function mobile_number_validation()
{
var mobile_no_format=/^\d{10}$/;
inputtxt=document.getElementById("mobile_no").value;
Mobile no:<input id="mobile_no">
<button type="button" onclick="mobile_number_validation()">submit
</button>
</body>
</html>
```

d) Validates that a string has not all blank /whitespace characters:

```
<html>
```

```
<body>
<script>
function string_empty_validation()
{
x = document.getElementById("str").value;
x=x.trim();
if (x.length == 0)
{
alert("string has blank spaces only");
return false;
}
else
alert("success");
return true;
}
</script>
input string: <input id="str">
<button type="button" onclick="string_empty_validation()">Submit</button>
</body>
</html>
```

e) Validating string for minimum length

```
<html>
<body>
<script>
function string_length_validation()
{
string=document.getElementById("str").value;
if(string.length < 4)
{
alert("minimum length ")
return true;
}
else
{
alert("minimum length of sting is 4")
return false;
}
}
</script>
input string:<input id="str">
```



```
<button type="button" onclick="string_length_validation()">Submit</button>
</body>
```

f) Checking string for all letters

```
<html>
<body>
<script>
function all_letter_validation()
{
letters=/^[A-Za-z]+$;/
string=document.getElementById("str").value;
if(string.match(letters))
{
alert("correct");
return true;
}
else
{
alert("enter only A-Z or a-z characters");
return false;
}
}
</script>
input string:<input id="str" >
<button type="button" onclick="all_letter_validation()">Submit</button>
</body>
</html>
```

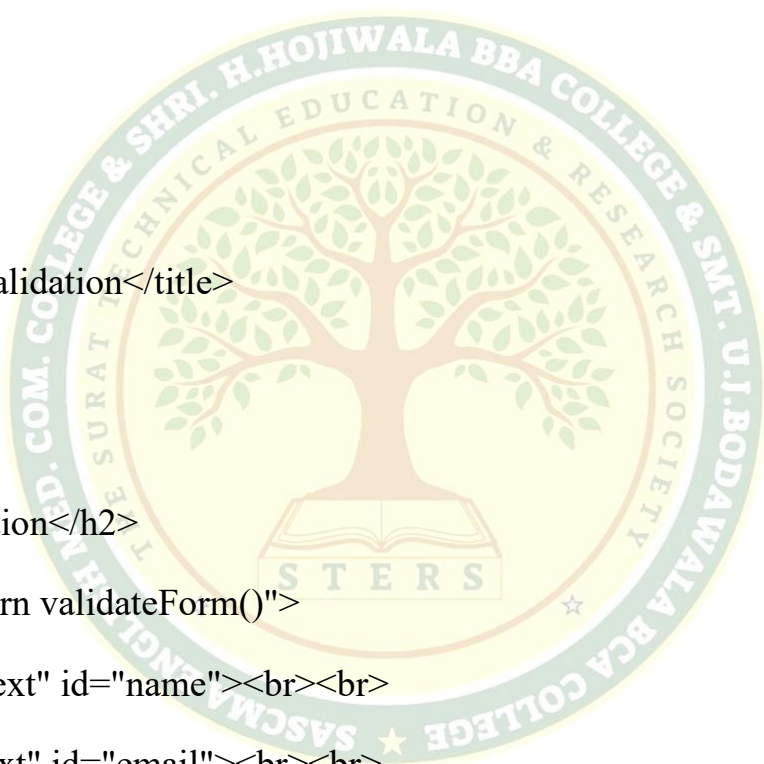
g) Validating name

```
<html>
<body>
<script>
function name_validation()
{
var regName = /^[a-zA-Z][a-zA-Z]+$/;
string=document.getElementById("str").value;
if(string.match(regName))
{
alert('valid name given');
}
else {
alert('In Valid name given');
}
```

```
}  
}  
</script>  
input name:<input id="str">  
<button type="button" onclick="name_validation()"> Submit </button>  
</body>  
</html>
```

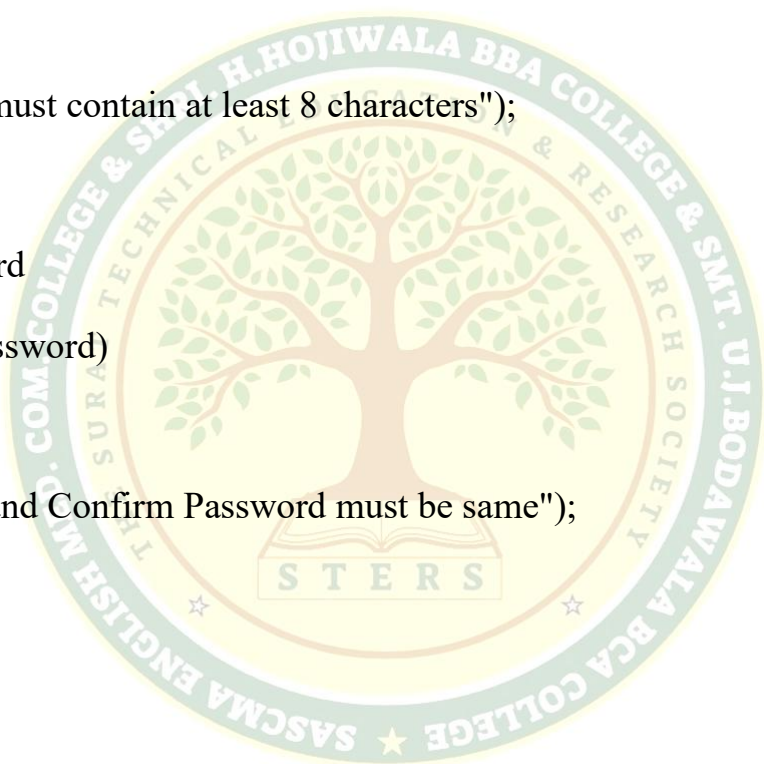
Student Registration Form

```
<!DOCTYPE html>  
  
<html>  
  
<head>  
  
  <title>JavaScript Validation</title>  
  
</head>  
  
<body>  
  
  <h2>Student Registration</h2>  
  
  <form onsubmit="return validateForm()">  
  
    Name: <input type="text" id="name"><br><br>  
  
    Email:<input type="text" id="email"><br><br>  
  
    Age:<input type="text" id="age"><br><br>  
  
    Password:<input type="password" id="password"><br><br>  
  
    Confirm Password:<input type="password" id="cpassword"><br><br>  
  
    Gender:<input type="radio" name="gender" value="Male"> Male  
  
    <input type="radio" name="gender" value="Female"> Female<br><br>
```



```
<input type="checkbox" id="terms">I agree to the terms and conditions<br><br>
<input type="submit" value="Register">
</form>
<script>
function validateForm(){
    var name = document.getElementById("name").value;
    var email = document.getElementById("email").value;
    var age = document.getElementById("age").value;
    var password = document.getElementById("password").value;
    var cpassword = document.getElementById("cpassword").value;
    if (name.trim() == "")// Name validation
    {
        alert("Please enter your name");
        return false;
    }// Email validation
    var emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (!emailPattern.test(email))
    {
        alert("Please enter a valid email");
        return false;
    }// Age validation
```

```
if (age == "" || isNaN(age) || age < 18)
{
    alert("Please enter a valid age");
    return false;
} // Password validation
if (password.length < 8)
{
    alert("Password must contain at least 8 characters");
    return false;
} // Confirm password
if (password != cpassword)
{
    alert("Password and Confirm Password must be same");
    return false;
}
// Gender validation
var gender = document.querySelector('input[name="gender"]:checked');
if (gender == null)
{
    alert("Please select gender");
    return false;
}
```



```

} // Terms validation

if (!document.getElementById("terms").checked)

{

    alert("Please accept terms and conditions");

    return false;

}

alert("Registration Successful");

return true;

}

</script>

</body>

</html>

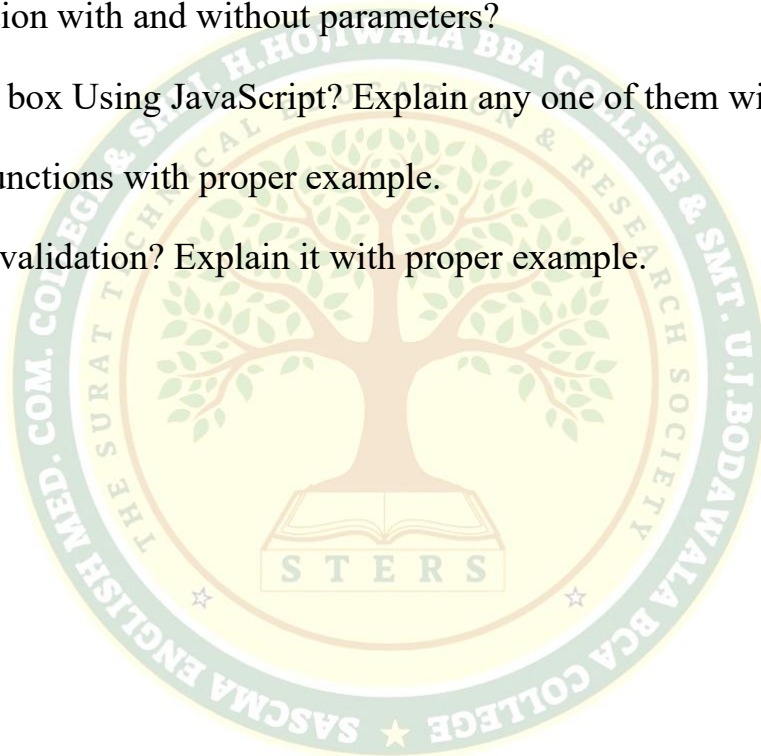
```

Email – URL pattern Description

Part	Meaning
/ ... /	Start and end of the regular expression
^	Start of the string
[^\s@]+	One or more characters that are not space and not @ Here \s means whitespace character.
@	Must contain @
[^\s@]+	One or more characters that are not space and not @
\.	A literal dot.
[^\s@]+	One or more characters that are not space and not @
\$	End of the string

Previous Years' Veer Narmad South Gujarat University Examination Questions

- 1.Explain all string function of JavaScript with example. [nov-2024]
- 2.Explain different loops in JavaScript. [march-2025]
- 3.discuss the difference between client-side and server-side scripting. [nov-2024]
4. List out all mouse events? Explain any 4 of them in detail. [march-2025]
5. Explain form and key events of JavaScript. [internal-2025]
6. How to define function with and without parameters?
- 7.how to create dialog box Using JavaScript? Explain any one of them with example.
- 8.Explain all strings functions with proper example.
- 9.What is data format validation? Explain it with proper example.





Unit 4 : JavaScript Objects

Unit 4: JavaScript Objects

[As per VNSGU Syllabus] – Year 2025-2026



Index

	Content	Page No
4.1	Creating Object	
4.2	Data Object (Date Constructor, Date Methods)	
4.3	Document Object Model (DOM)	
-	Previous Years' VNSGU Exam Questions	



4.1 Creating Object

Creating Object by Object Literal

This is the easiest way to create a JavaScript object. Using an object literal, you both define and create an object in one statement. Spaces and line breaks are not important, and an object definition can span multiple lines. The example below creates a JavaScript object and then adds properties to it.

```
<p id="demo"></p>
<script>
const person = {firstName:"John", lastName:"Doe", age:50, eyeColor:"blue"};
document.getElementById("demo").innerHTML =
  person.firstName + " is " + person.age + " years old.";
</script>
```

An empty object can also be created first, and properties added to it one at a time:

```
<p id="demo1"></p>
<script>
const person1 = {};
person1.firstName = "John";
person1.lastName = "Doe";
person1.age = 50;
person1.eyeColor = "blue";
document.getElementById("demo1").innerHTML =
  person1.firstName + " is " + person1.age + " years old.";
</script>
```

Creating Object by Using an Object Constructor

It is considered good practice to name constructor functions with an upper-case first letter. Inside a constructor function, the keyword `this` does not have a value of its own — it is a substitute for the new object being created, and it becomes that new object once the object is actually created with the `new` keyword.

```
<p id="demo"></p>
<script>
// Constructor function for Person objects
function Person(first, last, age, eye) {
  this.firstName = first;
  this.lastName = last;
  this.age = age;
  this.eyeColor = eye;
}
```



```
// Create a Person object
const myFather = new Person("John", "Doe", 50, "blue");

// Display age
document.getElementById("demo").innerHTML = "My father is " + myFather.age + ".";
</script>
```

Once a constructor function is defined, any number of objects of the same type can be created by calling that constructor with the new keyword:

```
// Create two Person objects
const myFather = new Person("John", "Doe", 50, "blue");
const myMother = new Person("Sally", "Rally", 48, "green");

document.getElementById("demo").innerHTML =
  "My father is " + myFather.age + ". My mother is " + myMother.age + ".";
```

4.2 Data Object

4.2.1 Date Constructor

Date() — When a Date object is created, a number of methods allow you to operate on it. Date methods allow you to get and set the year, month, day, hour, minute, second and millisecond of date objects, using either local time or UTC (Coordinated Universal Time / Greenwich Mean Time). When a date object is displayed in HTML it is automatically converted to a string using the `toString()` method.

```
const d = new Date();
document.getElementById("demo").innerHTML = d.toString();

const d1 = new Date();
document.getElementById("demo1").innerHTML = d1.toUTCString();

const d2 = new Date();
document.getElementById("demo2").innerHTML = d2.toDateString();

const d3 = new Date();
document.getElementById("demo3").innerHTML = d3.toISOString();

Date(milliseconds) — JavaScript stores dates internally as the number of milliseconds that have elapsed since January 1, 1970, 00:00:00 UTC. For example, 1000000000000 milliseconds from that reference point is approximately March 3, 1973:

const d = new Date(1000000000000);
```

```
document.getElementById("demo").innerHTML = d;
```

Date(year, month, day, hours, minutes, seconds, milliseconds) — a Date object can also be built from its individual components, and read back with getter methods such as getFullYear(), getMonth() and getDate():

```
const d = new Date();
```

```
document.getElementById("demo").innerHTML = d.getFullYear();
```

```
const d1 = new Date();
```

```
document.getElementById("demo1").innerHTML = d1.getMonth() + 1;
```

```
const d2 = new Date();
```

```
document.getElementById("demo2").innerHTML = d2.getDate();
```

4.2.2 Date Methods

Method	Description
getFullYear()	Get the year as a four digit number (yyyy)
getMonth()	Get the month as a number (0–11)
getDate()	Get the day as a number (1–31)
getHours()	Get the hour (0–23)
getMinutes()	Get the minute (0–59)
getSeconds()	Get the second (0–59)
getMilliseconds()	Get the millisecond (0–999)
getTime()	Get the time (milliseconds since January 1, 1970)
getDay()	Get the weekday as a number (0–6)

getDate(), getDay(), getMonth() and getHours() examples:

```
const d = new Date();
```

```
document.getElementById("demo").innerHTML = d.getDate(); // day of month
```

```
const d1 = new Date();
```

```
document.getElementById("demo1").innerHTML = d1.getDay(); // weekday number
```

```
const d2 = new Date();
```

```
document.getElementById("demo2").innerHTML = d2.getMonth() + 1;
```

```
const d3 = new Date();
```

```
document.getElementById("demo3").innerHTML = d3.getHours();
```

setDate() and setMonth() change the value of an existing Date object:

```
const d = new Date();
```

```
d.setDate(15);
document.getElementById("demo").innerHTML = d;

const d2 = new Date();
d2.setMonth(4);
document.getElementById("demo2").innerHTML = d2;
toString() converts a Date object to a readable string:

const d = new Date();
let text = d.toString();
document.getElementById("demo").innerHTML = text;
```

4.3 Document Object Model (DOM)

4.3.1 DOM Concepts

With the HTML DOM, JavaScript can access and change every element of an HTML document. When a web page loads, the browser builds a Document Object Model of the page: a tree of objects rooted at Document, with the <html> root element branching into <head> and <body>, and every tag, attribute and piece of text represented as a node in that tree.

With this object model, JavaScript has all the power it needs to create dynamic HTML. It can:

- change all the HTML elements and attributes in the page
- change all the CSS styles in the page
- remove existing HTML elements and attributes, and add new ones
- react to existing HTML events in the page, and create new ones

4.3.2 DOM Properties

These are some typical DOM properties, where x is a node object (an HTML element):

Property	Description
x.innerHTML	The inner text value of x
x.nodeName	The name of x
x.nodeValue	The value of x
x.parentNode	The parent node of x
x.childNodes	The child nodes of x
x.attributes	The attribute nodes of x

4.3.3 DOM Methods

write() — writes directly to an open (HTML) document stream:

```
<script>
document.write("<h2>Hello World!</h2><p>Have a nice day!</p>");
</script>
```

writeln() — like write(), but adds a new line (CR) after each statement:

```
<pre>
<script>
document.writeln("Hello World!");
document.writeln("Have a nice day!");
</script>
</pre>
```

getElementById() — returns the element with the specified id, or null if it does not exist. It is one of the most commonly used methods in the HTML DOM, used almost every time an HTML element needs to be read or edited:

```
<h1 id="demo">The Document Object</h1>
<script>
const myElement = document.getElementById("demo");
myElement.style.color = "red";
</script>
```

getElementsByName() — returns a collection of all elements that share the specified name attribute:

```
<p>First Name: <input name="fname" type="text" value="Michael"></p>
<p id="demo"></p>
<script>
let elements = document.getElementsByName("fname");
document.getElementById("demo").innerHTML = elements[1].tagName;
</script>
```

Previous Years' VNSGU Examination Questions

1. Explain Date Object. Explain getDate(), getDay(), getMonth(), setDate(), setMonth(), setDay() date methods.
2. What is DOM? Explain writeln(), getElementById(), getElementsByName() DOM methods.
3. Explain creating an object using: (a) Object literal, (b) Object constructor.
4. What is Document Object Model? Explain its methods in detail.
5. Explain dialog boxes in JavaScript.